arXiv:2601.14192v1 [cs.AI] 20 Jan 2026

Toward Efficient Agents: A Survey of Memory, Tool learning, and Planning

Xiaofang Yang[1,2,†] **Lijun Li**[1,†,✉] **Heng Zhou**[1,3,†] **Tong Zhu**[1,†] **Xiaoye Qu**[1] **Yuchen Fan**[1,4] **Qianshan Wei**[5] **Rui Ye**[4] **Li Kang**[1,4] **Yiran Qin**[6] **Zhiqiang Kou**[7] **Daizong Liu**[8] **Qi Li**[5] **Ning Ding**[9] **Siheng Chen**[4] **Jing Shao**[1,✉]

[1] Shanghai Artificial Intelligence Laboratory, [2] Fudan University,
[3] University of Science and Technology of China, [4] Shanghai Jiaotong University,
[5] Institute of Automation, Chinese Academy of Sciences,
[6] The Chinese University of Hong Kong (Shenzhen),
[7] Hong Kong Polytechnic University, [8] Wuhan University, [9] Tsinghua University

Abstract:

Recent years have witnessed increasing interest in extending large language models into agentic systems. While the effectiveness of agents has continued to improve, **efficiency**, which is crucial for real-world deployment, has often been overlooked. This paper therefore investigates efficiency from three core components of agents: **memory, tool learning, and planning**, considering costs such as latency, tokens, steps, etc. Aimed at conducting comprehensive research addressing the efficiency of the agentic system itself, we review a broad range of recent approaches that differ in implementation yet frequently converge on **shared high-level principles** including but not limited to bounding context via compression and management, designing reinforcement learning rewards to minimize tool invocation, and employing controlled search mechanisms to enhance efficiency, which we discuss in detail. Accordingly, we characterize efficiency in two complementary ways: comparing effectiveness under a fixed cost budget, and comparing cost at a comparable level of effectiveness. This trade-off can also be viewed through the Pareto frontier between effectiveness and cost. From this perspective, we also examine efficiency oriented benchmarks by summarizing evaluation protocols for these components and consolidating commonly reported efficiency metrics from both benchmark and methodological studies. Moreover, we discuss the key challenges and future directions, with the goal of providing promising insights.

† *Main contributors*
✉ *Corresponding Author*

Keywords: Agents, Efficiency, Agent Memory, Tool Learning, Planning

🗓️ **Date**: January 20th, 2026
🏠 **Projects**: https://efficient-agents.github.io/
Code Repository: https://github.com/yxf203/Awesome-Efficient-Agents
✉ **Contact**: yangxiaofang@pjlab.org.cn, lilijun@pjlab.org.cn, hengzzzhou@gmail.com, zhutong@pjlab.org.cn

Contents

1 Introduction **4**

2 Preliminaries **5**

2.1 Agent Formulation 5

2.2 From Pure LLMs to Agents 5

3 Efficient Memory **6**

3.1 Memory Construction 7

3.1.1 Working Memory 8

3.1.2 External Memory 10

3.2 Memory Management 12

3.2.1 Rule-based Management 12

3.2.2 LLM-based Management 13

3.2.3 Hybrid Management 14

3.3 Memory Access 15

3.3.1 Memory Selection 15

3.3.2 Memory Integration 16

3.4 Multi-Agent Memory 17

3.5 Discussion 19

4 Efficient Tool Learning **20**

4.1 Tool Selection 21

4.2 Tool Calling 23

4.3 Tool-Integrated Reasoning 25

4.4 Discussion 26

5 Efficient Planning **27**

5.1 Single-Agent Planning Efficiency 27

5.2 Multi-Agent Collaborative Efficiency 29

5.3 Discussion 30

6 Benchmarks **30**

6.1 Memory 31
6.2 Tool Learning 31
6.3 Planning 33

7 Challenges and Future Directions **34**

8 Conclusion **35**

1. Introduction

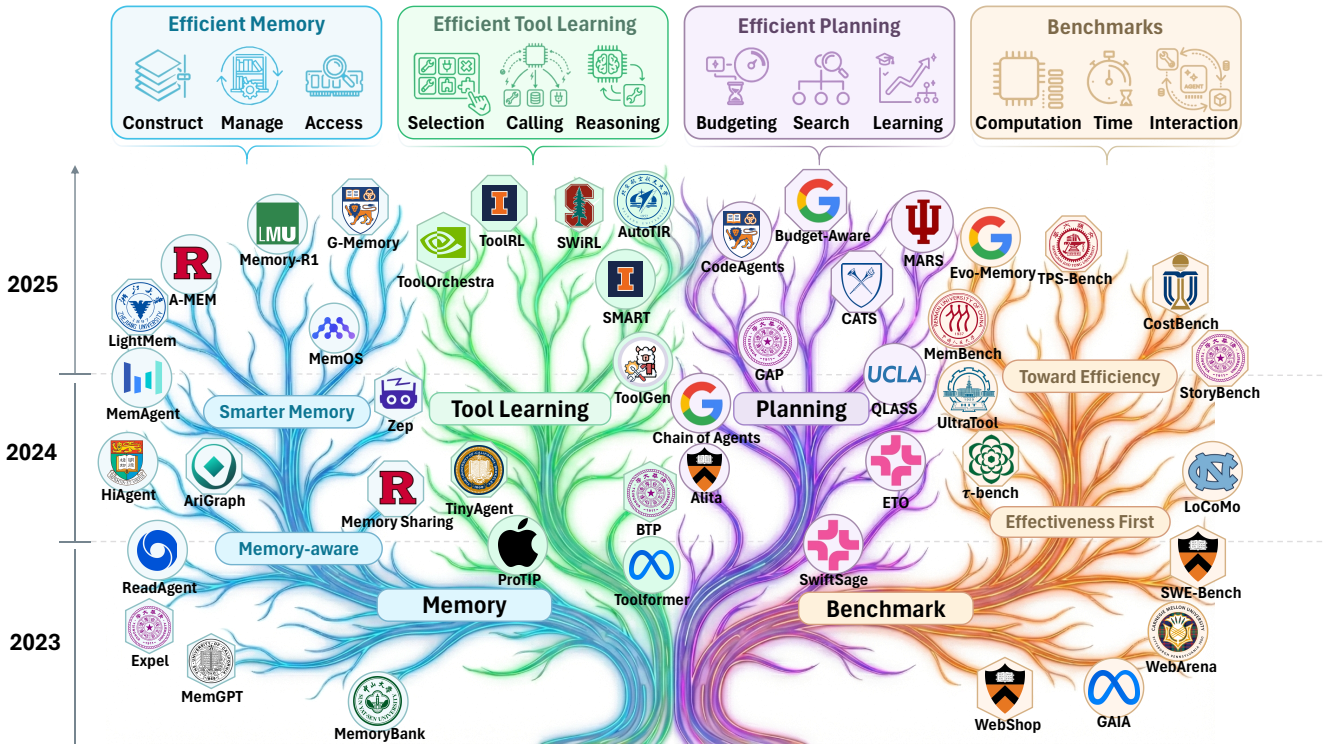


Figure 1: The evolutionary trajectory of efficient agent research. The diagram is organized into four principal branches: Memory, Tool Learning, Planning, and Benchmarks. Key works and their institutional affiliations are mapped chronologically to illustrate the field's development and categorization from 2023 to 2025.

The landscape of Artificial Intelligence has undergone a paradigm shift, evolving from the era of Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to the advent of Large Language Models (LLMs), and the emergence of LLM-based Agents currently [53, 27, 107, 44, 171, 37]. Unlike their predecessors, which primarily focused on perception or static text generation, agentic systems do not merely process information; they actively interact with external environments to execute complex, multi-step workflows across diverse domains, such as autonomous software engineering [183, 166] and accelerated scientific discovery [161, 75, 29].

However, this shift toward autonomous action has introduced a critical bottleneck: **efficiency**. While the deployment of LLMs is already resource-intensive, this challenge is **significantly exacerbated** in agentic systems. Unlike a standard LLM that typically operates in a linear, single-turn query-response format, an agent consumes exponentially more resources due to its recursive nature. To automate intricate real-world tasks [38, 34, 82, 166], agents must perform extensive memory management, iterative tool usage, and complex planning over multiple steps. This multi-step execution leads to prohibitive latency, context window saturation, and excessive token consumption, raising profound concerns regarding the long-term sustainability and equitable accessibility of these increasingly capable systems.

To understand the urgency of agent efficiency, one must examine the typical agentic workflow. Upon receiving a user instruction, an agent engages in a recursive loop that heavily uses the following key

components: memory, planning, and tool learning to observe output and provide the final solution.

$$\text{Input} \rightarrow \Big[\underbrace{\text{Memory}}_{\text{Context}} \rightarrow \underbrace{\text{Planning}}_{\text{Decision}} \rightarrow \underbrace{\text{Tool Learning}}_{\text{Action}} \rightarrow \underbrace{\text{Observation}}_{\text{Feedback}}\Big]_n \rightarrow \text{Solution}.$$

In each iteration n, the system must first retrieve relevant context from memory, reason over the current state to formulate a plan, execute a specific tool-incorporated action, and process the resulting observation. This cycle creates a compounding accumulation of tokens, where the output of step n becomes the input cost of step $n+1$, resulting in high inference costs and slow response times. Consequently, mere model compression is insufficient. We therefore define an efficient agent as follows:

Efficient agent is not a smaller model, but as an agentic system optimized to maximize task success rates while minimizing resource consumption, including token usage, inference latency, and computational cost across memory, tool usage, and planning modules.

Our survey aims to systematize the numerous efforts in this emerging field. While a large number of existing surveys focus on Efficient LLMs [156, 201, 123], which serve as the backbone of agents, there is a lack of comprehensive literature addressing the efficiency of the agentic system itself. To bridge this gap, we categorize existing works into three strategic directions: 1) Efficient Memory: Techniques for compressing historical context, managing memory storage, and optimizing context retrieval. 2) Efficient Tool Learning: Strategies to minimize the number of tool calls and reduce the latency of external interactions. 3) Efficient Planning: Strategies to reduce the number of executing steps and API calls required to solve a problem.

The remainder of this survey is organized as follows: Section 2 introduces the preliminaries and highlights the efficiency gap between agents and LLMs. Sections 3 through 5 explore component-level efficiency, with a focus on memory, tool learning, and planning optimizations. Subsequently, Section 6 addresses the quantification of efficiency. The survey concludes with a discussion on open challenges and future research directions.

2. Preliminaries

2.1. Agent Formulation

We model an LLM-based agent interacting with an environment as a partially observable Markov decision process (POMDP) augmented with an external tool interface and an explicit memory component. Formally, we define the overall model as

$$\mathcal{M} = (\mathcal{S}, \mathcal{O}, \mathcal{A}, P, R, \gamma;\ \mathcal{T}, \Psi;\ \mathcal{M}_{mem}, U, \rho).$$

Here $\mathcal{S}$ denotes the latent environment state space, $\mathcal{O}$ the observation space, and $\mathcal{A}$ the agent action space. The environment dynamics are given by the transition kernel P, the reward function R, and the discount factor $\gamma \in [0,1)$.

The agent is additionally equipped with a set of external tools $\mathcal{T}$ and a tool interface Ψ, which specifies how tool calls are executed and what tool outputs are returned to the agent. Finally, we model explicit agent memory with memory state space $\mathcal{M}_{mem}$, an update rule U that maps the current memory and available information to the next memory state, and an initialization distribution ρ over the initial memory.

2.2. From Pure LLMs to Agents

We define efficiency through a cost–performance trade-off: achieving comparable performance with lower cost, or achieving higher performance under a similar cost budget.

We acknowledge that many efficiency techniques used in LLM-based agents overlap with those for standalone LLMs (e.g., model compression and inference acceleration). In agents, however, these techniques mainly serve as foundational enablers rather than addressing the agent-specific sources of inefficiency. As summarized by Wang et al. [126], compared to pure LLMs, LLM-based agents exhibit more human-like decision-making by augmenting a base model with cognitive components such as planning and memory.

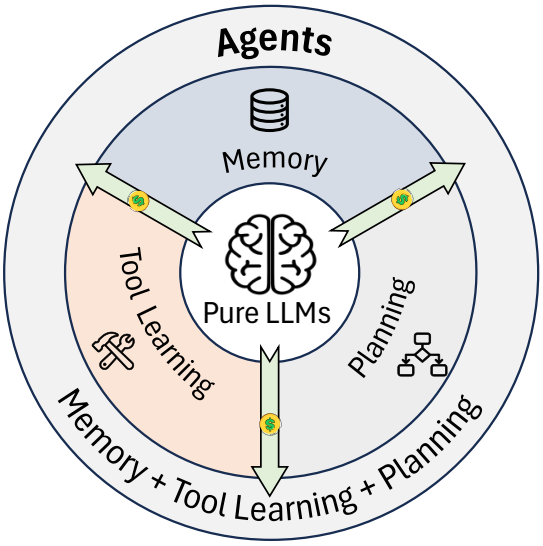


Figure 2: From LLMs to agents: standalone reasoning to trajectory-level reasoning with memory, planning, and tool learning, while introducing additional cost sources.

Accordingly, in this subsection we focus on what differentiates agent efficiency from LLM efficiency. From a functional perspective, an agent is characterized by its ability to (i) plan and act over multiple steps, (ii) invoke external tools or environment commands to acquire information and execute operations, and (iii) condition subsequent decisions on retrieved or updated memory.

As illustrated in Figure 2, agentic systems introduce additional cost sources beyond generation. For a pure LLM, the inference cost is often dominated by token generation and can be approximated as:

$$\mathrm{Cost}_{\mathrm{LLM}} \approx \alpha\, N_{\mathrm{tok}},$$

where N_{tok} is the number of generated reasoning tokens and α captures the per-token cost (e.g., time or monetary cost). In contrast, an agent may incur additional overhead from tools, memory, and retries as needed:

$$\mathrm{Cost}_{\mathrm{agent}} \approx \alpha\, N_{\mathrm{tok}} + \mathbb{I}_{\mathrm{tool}} \cdot \mathrm{Cost}_{\mathrm{tool}} + \mathbb{I}_{\mathrm{mem}} \cdot \mathrm{Cost}_{\mathrm{mem}} + \mathbb{I}_{\mathrm{retry}} \cdot \mathrm{Cost}_{\mathrm{retry}},$$

where $\mathbb{I}_{\mathrm{tool}}, \mathbb{I}_{\mathrm{mem}}, \mathbb{I}_{\mathrm{retry}} \in \{0,1\}$ are indicator variables that equal 1 if the agent invokes tools, accesses memory, or performs retries, respectively, and 0 otherwise. Therefore, improving agent efficiency is not only about reducing language generation, but also about reducing the frequency and improving the selectivity of tool or memory invocations and retries along a trajectory, to achieve a better cost–performance trade-off.

3. Efficient Memory

A major efficiency bottleneck for LLM agents is the computational and token overhead induced by long contexts and long-horizon interactions, where agents may repeatedly reprocess large histories to act. **Memory-augmented reasoning** provides a principled way to alleviate this inefficiency. By storing and reusing past experience, including successes, failures, and interaction traces, agents can avoid redundant computation, make more informed decisions, and reduce costly retries. In this sense, memory is not merely an auxiliary component. It is a key mechanism for improving the overall efficiency-effectiveness trade-off of agent systems.

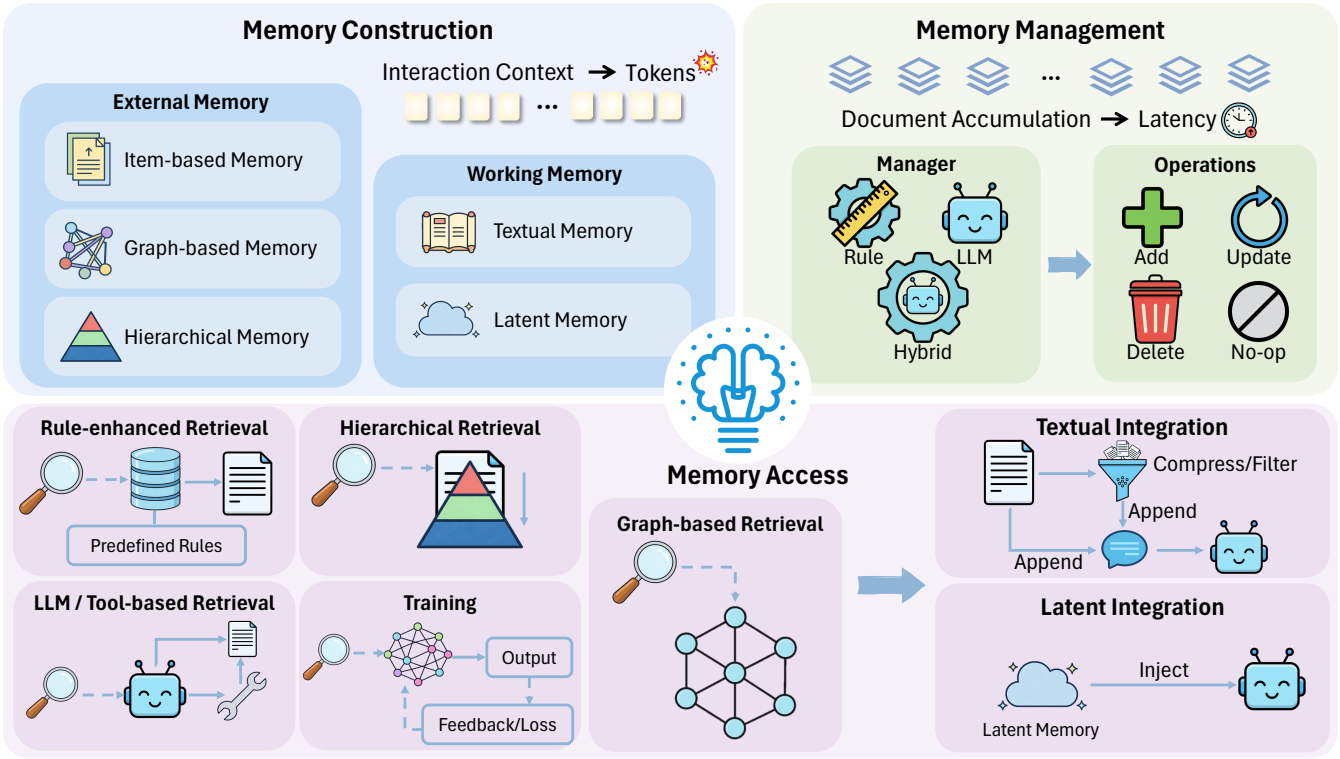


Figure 3: Efficient memory overview. This figure summarizes the agent-memory lifecycle in three phases: **Memory Construction**, which compresses long interaction context in working and external memory to mitigate token explosion; **Memory Management**, which curates and updates an accumulating memory store via rule-based, LLM-based, or hybrid strategies to control latency; and **Memory Access**, which determines what memories to retrieve and how to integrate them into the model.

We organize this section around the lifecycle of agent memory, covering memory construction, memory management, and memory access. Because memory is central to efficiency gains, how to design an efficient memory module becomes an important problem. We therefore discuss efficiency-oriented designs throughout this lifecycle, focusing on how to maximize the benefit of memory while minimizing additional overhead. Figure 3 provides a structured overview of our taxonomy, and Table 1 lists representative works for an at-a-glance summary.

3.1. Memory Construction

No matter whether we target long-context tasks or long-term interactions, the core challenge is handling extensive context or interaction history. Naively appending raw history into the prompt is often impractical: token usage grows rapidly, and performance can even degrade when relevant information is buried in long sequences, as observed in the "lost in the middle" phenomenon [71]. In addition, an LLM's context window is finite, whereas the amount of potentially relevant information is effectively unbounded. These constraints motivate memory construction, which compresses and organizes past information into more manageable representations. Many existing works build memory through summarization, reducing token consumption and improving efficiency.

Table 1: Memory overview of efficiency-oriented mechanisms. The table is organized according to the taxonomy proposed in this work, covering working memory, external memory, and multi-agent memory.

Method	Category	Core Mechanism	Resource Link
		Working Memory	
COMEDY [11]	Textual	Two-stage memory distillation	GitHub
MemAgent [176]	Textual	Overwrite fixed memory	GitHub
MEM1 [200]	Textual	Update a compact shared internal state	GitHub
AgentFold [175]	Textual	Proactive context folding	N/A
DC [116]	Textual	Persistent, evolving memory	GitHub
Activation Beacon [184]	Latent	Activation-level beacon for long context	GitHub
MemoRAG [94]	Latent	KV-compressed global memory representation	GitHub
MemoryLLM [133]	Latent	a fixed-size latent memory pool	GitHub
M+ [134]	Latent	Dual-level latent memory; co-trained retriever	GitHub
Memory^3 [165]	Latent	Externalize knowledge into retrievable sparse KV memories	N/A
Titans [7]	Latent	Sliding-window attention; test-time trainable neural long-term memory	N/A
MemGen [181]	Latent	On-demand latent memory synthesis	GitHub
		External Memory	
MemoryBank [194]	Item-based	Ebbinghaus forgetting curve–based memory management	GitHub
RECOMP [153]	Item-based	Compress retrieved documents	GitHub
Expel [192]	Item-based	experiential learning; insight distillation and management	GitHub
Human-like memory [39]	Item-based	Cue-triggered memory recall	N/A
SeCom [86]	Item-based	Segment-level memory; compression-based denoising for retrieval	GitHub
Memory-R1 [162]	Item-based	Adaptive memory CRUD and memory distillation, via two RL-trained agents	N/A
Mem0 [15]	Item-based	Extract candidate memories; memory CRUD	GitHub
agentic plan caching [186]	Item-based	Store plan template; plan cache lookup (hit/miss) and update	N/A
LD-Agent [57]	Item-based	Separate different memory; topic-based retrieval	GitHub
MemoChat [76]	Item-based	Structured on-the-fly memos	GitHub
RMM [118]	Item-based	Topic-based memory organization; consolidation (add/merge); online RL reranker	N/A
Memento [197]	Item-based	Parametric case retrieval via an online-updated Q-function	GitHub
MemInsight [103]	Item-based	Attribute-augmented memory; attribute-guided retrieval	GitHub
ReasoningBank [83]	Item-based	Distill strategies from failures and successes to cut exploration steps	N/A
A-MEM [157]	Item-based	Atomic structured notes; link generation and memory evolution	GitHub
ACE [185]	Item-based	Incremental delta updates, lightweight merge and de-dup	GitHub
Agent KB [119]	Item-based	Cross-framework reusable experience Knowledge Base	GitHub
GraphReader [60]	Graph-based	Graph-guided coarse-to-fine exploration	N/A
KG-Agent [46]	Graph-based	Tool-based hop-local KG processing	N/A
Zep [99]	Graph-based	Temporal KG memory	GitHub
Mem0^g [15]	Graph-based	Extract candidate nodes; graph updation	GitHub
AriGraph [5]	Graph-based	Memory graph; semantic-to-episodic cascading retrieval	GitHub
D-SMART [55]	Graph-based	Structured OWL-compliant KG	N/A
MemGPT [84]	Hierarchical	OS-style virtual memory paging for context	Website
MemoryOS [49]	Hierarchical	OS-inspired three-tier memory hierarchy with policy-based inter-tier updates	GitHub
MemOS [63]	Hierarchical	Policy-guided type transformation of MemCubes across three memory forms	GitHub
ReadAgent [54]	Hierarchical	Gist memory compression; on-demand lookup	GitHub
HiAgent [40]	Hierarchical	Subgoals as memory chunks; on-demand trajectory retrieval	GitHub
H-MEM [115]	Hierarchical	Layer-by-layer retrieval	N/A
LightMem [21]	Hierarchical	Pre-compression; soft update (test-time); sleep-time update (offline)	GitHub
		Multi-Agent Memory	
MS [24]	Shared	Shared memory pool; selective addition; continual retriever training	GitHub
G-Memory [180]	Shared	three-tier graph memory with bi-directional coarse-to-fine retrieval	GitHub
RCR-Router [69]	Shared	Feedback-refined iterative router under a token budget	N/A
MemIndex [104]	Shared	Intent-indexed bipartite graphs; semantic slicing and dynamic indexing	N/A
MIRIX [132]	Shared	Six-module hierarchical memory with staged retrieval and parallel updates	GitHub
Intrinsic Memory Agents [177]	Local	Role-aligned templates; intrinsic iterative updates	N/A
AgentNet [167]	Local	Fixed-size memory modules for routing/execution; dynamic pruning	GitHub
DAMCS [164]	Local	Decentralized per-agent STWM/LTM with goal-oriented hierarchical knowledge graph	Website
SRMT [102]	Mixed	Personal latent memory and globally broadcast shared recurrent memory	GitHub
Collaborative Memory [100]	Mixed	Policy-based filtering/transformation of memory fragments; shared-memory reuse	N/A
LEGOMem [31]	Mixed	Role-aware memory routing; runtime-efficient retrieval scheduling	N/A

3.1.1. Working Memory

Working memory is the information directly available at inference time that conditions generation. Here, the term is broader than the common definition that limits working memory to context tokens. It includes

the text currently present in the prompt or context window, and latent memory in the form of continuous signals that influence the forward computation without being represented as tokens, such as soft prompts, KV cache, and hidden states. Latent memory can arise inside the model or be stored externally and injected as continuous conditioning. Embeddings count as latent memory only when they are provided to the model as such conditioning signals; embeddings used only to support retrieval are treated separately in Section 3.1.2.

Textual Memory. In LLM-based agents, textual memory is a common instantiation of working memory. To address the long-context challenge, many methods aim to keep the working memory in the prompt at a roughly constant size. In practice, this is often achieved by frequently **rewriting or compressing** the memory as the process evolves.

COMEDY [11] uses an LLM to generate and compress memory: it extracts session-specific memories from past conversations and then condenses them into a compact representation of key events, the user profile, and relationship changes. MemAgent [176] and MEM1 [200] both process long inputs sequentially by rewriting and updating a compact memory state at each step: MemAgent updates a summarized memory after each chunk, while MEM1 uses reinforcement learning [182] to maintain a fixed-length internal state tagged by <IS></IS> that replaces itself in the next prompt. AgentFold [175] proactively folds interaction history into multi-scale summaries plus the latest full turn, slowing critical information loss while reducing token usage.

By retaining a compact memory in the prompt rather than the full history, these methods reduce the effective context length the LLM needs to attend to, thereby improving long-context performance while decreasing computational cost and increasing efficiency.

Latent Memory. Besides textual working memory, recent work also lets an agent keep its state in latent form, such as hidden activations or KV caches. This kind of memory is not shown as text, but it can be read and updated by the model. In many cases it is much cheaper than storing and re-reading the full interaction history, and thus is attractive for efficient agents.

One group of methods builds **compact latent memory** by compressing long contexts into a small set of activations in KV space. Activation Beacon [184] partitions the context into chunks and fine-grained units, interleaves beacon tokens by a compression ratio, and uses progressive compression to distill layer-wise KV activations into the beacons, which are accumulated as latent memory while raw-token activations are discarded. MemoRAG [94] performs memory formation by inserting memory tokens after each window as information carriers of global memory in KV space, and updating a memory-token KV cache across windows with separate weight matrices; the compact global memory can later be reused (e.g., as retrieval clues).

A second group maintains an **external pool of latent memory** and integrates it into the backbone LLM via attention at inference time, enabling reuse of stored information across steps and episodes. MemoryLLM [133] maintains a fixed-size memory pool of memory tokens updated via self-update, enabling reuse of stored latent knowledge without lengthening the prompt. M+ [134] adds a GPU/CPU two-tier long-term memory with a co-trained retriever that fetches only a few relevant memory tokens per layer, and Memory^3 [165] encodes a KB as sparse explicit key–value memory injected into attention at decoding time to avoid repeated document reading.

A third group lets latent memory be **a separate neural module** that can learn online together with the agent. Titans [7] builds latent memory by updating a neural memory module at test time, writing only when prediction error is high and skipping updates otherwise. MemGen [181] constructs latent memories via an

RL-trained memory trigger and a memory weaver that produces compact latent memory tokens as the stored representation.

Strictly speaking, some of the methods above are proposed as general memory modules for LLMs rather than full agent frameworks. However, from the view of efficient agent memory, they play the same role: they compress long interaction histories into compact latent states, update these states only when needed, and expose them to the policy through attention or simple interfaces. This allows an agent to keep and reuse long-horizon information without replaying the entire textual trajectory at each step.

Working Memory

- **Advantages**: The working memory is directly conditioned upon during generation, eliminating the latency and overhead associated with external retrieval or repeated encoding.
- **Disadvantages**: Expanding the working set leads to computational growth for textual memory or increased memory footprint for latent states, and risks performance degradation due to information dilution in long contexts.
- **Applicable Scenarios**: Textual memory is best for logic-heavy tasks within moderate context limits, while latent memory suits efficiency-critical applications requiring the reuse of historical states without re-processing.

3.1.2. External Memory

External memory refers to information stored outside the model in token-level form, including document collections, knowledge graphs, and retrieval systems such as RAG. It does not condition generation directly. Instead, it is accessed through retrieval and then expressed as tokens placed into the prompt or context window.

Item-based Memory. Early agent-memory systems often store full trajectories or experiences, sometimes alongside summaries, which leads to long context and inefficiency. MemoryBank [194] stores daily conversation records and summarizes past events and user profiles from these conversations, but it incurs high token costs. Similarly, Expel [192] suffers from a similar limitation, as it accumulates experiences through trial and error and distills them into natural-language insights.

To be more efficient, some works adopt methods such as **memory extraction, compress or summarization** to directly reduce context length. This way thereby lowers input token consumption while yielding a shorter but more informative context. Human-like memory [39] extracts episodic memories from users dialogues, encapsulating content and temporal context into database structure. SeCom [86] uses segmentation model to divide long-term conversations into topic-coherent segmentation, and applies the compress model to denoise the segmentation, which further promotes efficient retrieval. Memory-R1 [162] and Mem0 [15] both extract and summarize ongoing dialogue into candidate memories for downstream updating; Memory-R1 does so at each turn, while Mem0 forms candidate memories from the new message pair, using a conversation summary and recent messages as context. Agentic plan caching [186] turns a successful run into a reusable cache entry by rule-based filtering the execution log and then using a lightweight LLM to remove context-specific details, storing the result as a (keyword, plan template) pair. LD-Agent [57] separates event and persona memory, using short-term and long-term banks for timestamped dialogue context and embedded event summaries, and a persona extractor to store user and agent traits in long-term persona banks.

Beyond the extraction and compression strategies discussed above, another way to improve efficiency is

to **design more structured memory systems**. Organizing memory more systematically can enable faster retrieval, better utilization of stored information, and improved overall performance. And one type of the structured memory system is topic-indexed memories, which organizes interactions into topic-level groups and stores each topic summary together with its corresponding dialogue segment for efficient retrieval. MemoChat [76] and RMM [118] both build topic-indexed memories. MemoChat records topic–summary–dialogue entries on the fly, while RMM groups each session by topic and stores each topic summary with its corresponding dialogue segment. Then, some system constructs the attribute-annotated memory items by enriching each interaction with structured attributes such as LLM-mined attribute value pairs, contextual descriptions, keywords, and tags to support fine-grained retrieval. MemInsight [103] and A-MEM [157] both enrich raw interactions with structured attributes for retrieval: MemInsight annotates memories with LLM-mined attribute–value pairs, while A-MEM converts each interaction into an atomic note with LLM-generated contextual descriptions, keywords, and tags. Besides, a typical way is to distill experience libraries for reusable decision by summarizing trajectories or execution logs into standardized experience entries that capture reusable strategies, domain concepts, and common failure modes for retrieval and reuse. ReasoningBank [83] summarizes successful and failed trajectories into structured memory items with a title, brief description, and content, and stores them with the task query and trajectory for embedding-based retrieval. ACE [185] represents context as structured, itemized bullets, each with a unique identifier, counters tracking how often it was marked helpful or harmful, and content such as a reusable strategy, domain concept, or common failure mode. Agent KB [119] turns execution logs into structured experience entries through human-guided abstraction, using few-shot prompting and a standardized cross-framework action vocabulary.

Graph-based Memory. It's obvious that graph-based memory is also a structured memory form. Some methods focus on constructing graph-structured representations from long inputs or KG interactions, so multi-hop evidence can be organized and accessed efficiently. Targeted at the long context task, GraphReader [60] segments long text into chunks, compresses them into key elements and atomic facts, and uses these to construct a graph that captures long-range dependencies and multi-hop relations. KG-Agent [46] constructs a task-specific subgraph by tool calls and records the retrieved entities and relations as knowledge memory.

Another line of work constructs long-term memory directly as a dynamic knowledge graph, turning interactions into entities, relations, and time-aware facts that can be incrementally updated. Zep [99] builds memory as a temporally-aware knowledge graph by ingesting time-stamped episodes, extracting/aligning entities and relations, storing fact edges with periods of validity and additionally constructs a community subgraph that clusters strongly connected entities and stores high-level community summaries. Mem0^g [15] represents memory as a directed labeled graph, where an LLM converts new messages into entities and relation triplets that form candidate nodes or edges for graph updates. D-SMART [55] incrementally constructs an OWL-compliant dialogue KG by first distilling each turn into an assertion-like statement, then converting it into a KG fragment for integration. AriGraph [5] updates a unified semantic–episodic memory graph online by adding an episodic node for each observation and extracting triplets to update the semantic graph, linking the two via episodic edges.

Graph-based memory represents entities and their relations as a structured graph. Building the graph already compresses and normalizes the history by merging repeated content about the same entity into a single node and keeping only relevant relations as edges. This makes construction more efficient by producing a compact structure that avoids unbounded prompt growth and supports fast retrieval later.

Hierarchical Memory. Hierarchical memory organizes information into multiple linked levels, enabling coarse-to-fine, on-demand access. Most hierarchical memory methods consider both structure and content, but with different emphases. Accordingly, related work can be grouped by whether it places more weight on structural organization and management or on content abstraction and indexing.

System-oriented hierarchical memory designs define explicit storage tiers and read/write interfaces to manage long interaction history. MemGPT [84] constructs a hierarchical memory by partitioning the in-context prompt into system instructions, a writable working context, and a FIFO message buffer, and storing the remaining history and documents in external recall and archival memory. MemoryOS [49] adopts an OS-inspired hierarchical memory design with three storage tiers: short-term memory stores recent dialogue pages, mid-term memory groups pages into topic segments with summaries, and long-term personal memory maintains user and agent persona information. MemOS [63] standardizes memory as MemCubes, each composed of a structured metadata header and a memory payload that can encapsulate plaintext, activation states, or parameter deltas. New interactions are incrementally turned into MemCubes and organized in a hierarchical structure.

Content-oriented approaches build hierarchical indices by segmenting and compressing documents or trajectories into multi-granularity summaries. ReadAgent [54] splits a long document into pages and summarizes each page into a page-linked gist memory, forming a simple hierarchical index. HiAgent [40] compresses working memory into subgoals and observations, and stores full trajectories in external memory indexed by these summaries. H-MEM [115] constructs a hierarchical structure, with four memory layers: Domain Layer, Category Layer, Memory Trace Layer, and Episode Layer. It designs prompts to guide model to parse the interactions into these layers, which forms a progressively optimized index. LightMem [21] uses a sensory–STM–LTM pipeline that first pre-compresses inputs by the sensory module, then groups turns into topic segments for STM and periodically summarizes these segments into compact LTM entries.

External Memory

- **Advantages**: Effectively unbounded long-term storage outside the model, reducing context overflow via targeted retrieval.
- **Disadvantages**: Adds system overhead and retrieval latency, with potential retrieval noise.
- **Applicable scenarios**: Item-based memory suits general long-trajectory agents, graph-based memory suits entity–relation and multi-hop reasoning tasks, while hierarchical memory suits ultra-long histories or large corpora needing coarse-to-fine retrieval.

3.2. Memory Management

Some methods, like Human-like memory [39], continually insert new memories into memory module, without any operation like updating, removing or merging, leading to memory space explosion. Therefore, the speed of the memory retrieval or recall will significantly decrease, which indicates that memory management is a highly important part for efficiency.

3.2.1. Rule-based Management

Rule-based management refers to predefined rules for updating, removing, and merging existing memories. Because these rules are static, this approach is inexpensive and prevents the overall memory size from growing uncontrollably.

MemoryBank [194] introduces an Ebbinghaus-inspired memory update rule that decays memories over time while reinforcing important ones. Building on this idea, H-MEM [115] retains forgetting-curve-based decay and further adds feedback-driven regulation to dynamically adjust memory according to user feedback. Experimental results in A-MEM [157] suggest that forgetting-curve-based memory management effectively controls memory size and reduces retrieval time. However, it also leads to a substantial drop in task performance.

Apart from forgetting-curve-based policies, a common rule-based strategy is trigger-driven memory maintenance, such as evicting or migrating items when a fixed-size buffer reaches capacity (e.g., FIFO replacement) [84, 49]. In practice, these simple rules are often intertwined with LLM-based management, where the model summarizes or saves key information before items are removed or moved; more details are discussed in Section 3.2.3.

Rule-based Management

- **Advantages**: Fast, predictable, and low-cost memory management without extra LLM calls.
- **Disadvantages**: Static and task-agnostic rules can blindly prune or decay memory, causing critical information loss and hurting accuracy when retention matters.

3.2.2. LLM-based Management

LLM-based memory management can be broadly categorized by its decision form: selecting from a discrete set of operations versus generating open-ended updates.

A common formulation is operation selection, where the model **picks an action from a predefined set** (e.g., ADD/DELETE) and applies it to retrieved memories. Both Memory-R1 [162] and Mem0 [15] update an external memory by retrieving similar entries and choosing among ADD, UPDATE, DELETE, NOOP. Memory-R1 learns the choice via reinforcement learning, while Mem0 lets an LLM select the operation after vector-based retrieval. RMM [118] follows the same retrieve-then-update pattern: for each newly extracted topic memory, it retrieves the top-k most similar entries from the memory bank and prompts an LLM to decide whether to merge or add. Separately, ExpeL [192] maintains an insights list through direct list editing, applying operations such as ADD, EDIT, UPVOTE, and DOWNVOTE to correct or gradually suppress erroneous and outdated insights.

A different formulation casts memory management as **open-ended generation**, where the model produces the update itself and implicitly performs the update operation rather than picking from a fixed action set. A-MEM [157] uses generative updates: it retrieves top-k similar notes with a fixed encoder, then an LLM creates links and rewrites related notes via memory evolution.

LLM-based Management

- **Advantages**: Adaptive, task-aware decisions that keep the most relevant information while enabling effective compression or merging for a concise context.
- **Disadvantages**: Requires extra LLM calls during management, increasing compute cost and latency.

3.2.3. Hybrid Management

Hybrid memory management typically combines lightweight rule-based control with selective LLM-based operations to balance efficiency and effectiveness.

Typical designs include tier-specific management, where rule-based triggers promote or consolidate information across tiers and costly LLM updates are invoked only when necessary. MemoryOS [49] and LightMem [21] both adopt tier-specific, trigger-driven updates for hierarchical memory. MemoryOS manages STM as FIFO pages with overflow migrated to MTM, uses segment Heat scores in MTM for eviction and promotion, and updates LPM via an LLM, whereas LightMem triggers topic segmentation when the sensory buffer is full, summarizes topics into LTM when STM exceeds a token budget, and combines online soft updates with offline sleep-time consolidation. LD-Agent [57] uses a time-gap threshold as the trigger, summarizing the short-term cache into a long-term event record and clearing the cache to mark session boundaries. MemGPT [84] uses a hierarchical memory with main context and external context. A Queue Manager enforces token limits via memory pressure warnings, eviction, and recursive summarization, while a Function Executor turns model outputs into function calls to read and write across tiers.

Another management is item-level selection and pruning, using rules or heuristics for fast de-duplication and removal while relying on LLMs for semantic keep-or-drop decisions. Agent KB [119] and ACE [185] exemplify item-level selection and pruning for hybrid memory management. Agent KB reduces redundancy by thresholding embedding similarity and using an LLM ranker to keep the better experience, then evicts low-utility entries based on a learned utility score. ACE maintains a bulletized context through incremental delta updates and applies embedding-based grow-and-refine to merge, prune, and de-duplicate bullets, keeping the context compact.

Besides, some management also considers lifecycle policies that use lightweight metrics to schedule costly maintenance beyond tier transfer, such as consolidation, deduplication, and archiving. MemOS [63] manages MemCubes with explicit lifecycle and version tracking, using policy- and metric-driven modules such as MemScheduler and MemVault for deduplication, conflict handling, and archiving. Crucially, it supports type-aware transformation across Plaintext Memory, Activation Memory, and Parameter Memory, including promotion and demotion between types.

For graph-structured memory, hybrid management applies rule-based graph updates, while using LLMs to retrieve relevant subgraphs and verify contradictions or outdated content before updating relations. Zep [99], Mem0^g [15], and AriGraph [5] follow a similar pattern for graph memory maintenance: an LLM judges semantic conflicts or staleness against retrieved related edges, while the graph is updated through rule-based operations such as edge invalidation or removal and insertion of new relations to preserve temporal or world-model consistency. Additionally, D-SMART [55] maintains an OWL-compliant Dynamic Structured Memory and performs two-stage conflict resolution by letting an LLM identify contradicted or superseded triples, pruning them before merging the new fragment, with an optional OWL reasoner for logical consistency checking.

Hybrid Management

- **Advantages**: Balances low-cost, predictable rule control with task-aware LLM decisions, invoking the LLM only when needed to keep memory both efficient and relevant.
- **Disadvantages**: Increases system complexity across tiers, and can suffer from suboptimal policy interactions, while LLM calls still add cost and latency when invoked.

3.3. Memory Access

Memory access retrieves and uses only the small subset of a large memory bank that matters for a query, balancing retrieval latency and token cost against downstream generation quality.

3.3.1. Memory Selection

Memory selection determines what to retrieve and how to retrieve it. Most methods follow vanilla retrieval, i.e., encoding the query and its context into embeddings and selecting relevant information via similarity search, while others employ improved retrieval mechanisms to enhance retrieval quality and efficiency.

Rule-enhanced Retrieval. Some methods enhance retrieval by incorporating additional rule-based scoring factors and applying preprocessing steps before retrieval. Generative Agents [87] and Human-like memory [39] take time into consideration, namely recency and elapsed time in these works. Apart from this, Generative Agents adds importance, a score generated by LLM based on the semantic importance, and Human-like memory adds recall frequency, computed according to the mathematical model. Agent KB [119] employs a hybrid retrieval strategy that integrates lexical matching with semantic ranking by task similarity, combining both signals into a unified retrieval score. For long-term event retrieval, LD-Agent [57] combines semantic relevance, noun-based topic overlap, and an exponential time-decay factor into an overall score, and only retrieves memories whose semantic similarity exceeds a threshold. While the aforementioned methods improve retrieval by adding additional scoring factors while keeping the computational cost comparable to vanilla retrieval, MemInsight [103] augments memories with LLM generated attribute value annotations and leverages these augmentations for retrieval, either by filtering memories via attribute matching or by embedding the aggregated augmentations for vector similarity search.

Graph-based Retrieval. For graph-based memory, retrieval naturally follows the graph structure, enabling efficient neighbor expansion and more precise localization of relevant facts, especially when queries target entity- and relation-centric information. Given a textual query, Both AriGraph [5] and Mem0^g [15] retrieve from a memory graph by anchoring on query-relevant facts and expanding neighbors into a local subgraph. AriGraph retrieves semantic triplets and then ranks episodic vertices via episodic search, whereas Mem0^g pairs entity-centric subgraph construction with semantic triplet retrieval over relationship triplets.

LLM or Tool-based Retrieval. Furthermore, there are methods that do not depend on a retriever, but instead leverage LLMs or external tools for obtaining relevant information. For LLM-based retrieval, MemGPT [84] uses hierarchical memory without a fixed retrieval pipeline: memory tiers are exposed as tools, and the LLM selects the tier and operation under token budgets enforced by the system. MemoChat [76] exploits its memo structure by retrieving only the topic and summary, rather than the full topic–summary–dialogue, to reduce input length. ReadAgent [54] similarly delegates page lookup to the LLM, which decides when and which page(s) to consult. However, while using a strong LLM can improve retrieval accuracy, it often incurs substantial overhead in both token consumption and inference latency, making it more suitable for low-frequency, high-stakes queries where correctness outweighs cost. Besides, some methods rely on tool use for retrieval. GraphReader [60] predefines various tools, and employs the tools to read the memory step by step, from coarse-grained to fine-grained. D-SMART [55] lets the LLM select graph-operations such as Expand Entity and Find Path to retrieve n-hop neighbors from the global DSM and incrementally grow a task-specific subgraph, which serves as grounded context for answering.

Hierarchical Retrieval. In line with the hierarchical memory structure, retrieval can likewise be organized hierarchically. Some retrieval methods can be considered as a simple way of hierarchical retrieval, such as the conceptually two-layer design. HiAgent [40] can recall the trajectories by a retrieval module, when the agent needs to obtain the details of the previous subgoal. Beyond such a two-layer setup, hierarchical retrieval can be made explicit through multi-layer indexing. In H-MEM [115], each memory embedding points to relevant sub-memories in the next layer, recursively indexing down to the last layer to retrieve relevant information, thereby accelerating retrieval. At a more system level, MemoryOS [49] uses tier-specific retrieval: STM returns the most recent dialogue pages, MTM retrieves top-m candidate segments and selects top-k relevant pages within them, and LPM performs semantic search over long-term user and agent memories.

Training. As the memory bank grows, a fixed retriever can drift from what is truly useful, so recent work trains adaptive retrieval that prioritizes high-utility memories for better relevance and efficiency. RMM [118] adds a learnable reranker over a dense retriever and updates it online via RL using binary useful memory signals from Retrospective Reflection. Memento [197] learns a parametric Q-function over state–case pairs to rank and select Top-K cases, favoring historically high-reward cases over nearest neighbors.

Memory Selection

- **Applicable scenarios**: Rule-enhanced retrieval fits settings with clear heuristics or constraints and tight budgets; graph-based retrieval fits entity–relation queries and multi-hop evidence chaining; LLM/tool-based retrieval fits low-frequency, high-stakes queries where correctness outweighs latency; hierarchical retrieval fits very large memory banks requiring coarse-to-fine lookup; training-based retrieval fits long-running systems where the memory distribution drifts over time.

3.3.2. Memory Integration

Memory integration determines how to use retrieved content efficiently. It can leverage techniques such as filtering, compression, and structured insertion to make the retrieved information easier and cheaper to use during generation.

Textual Integration. When memory is stored as natural language, integration mainly means deciding which small set of text to show to the backbone model and in what format. DC-RS [116] integrates persistent memory by keeping a cheatsheet store, doing similarity-based retrieval, then synthesizing a compact cheatsheet that is inserted into the prompt.

Several agent-oriented systems follow the same idea but build on structured memory stores. In Mem0 [15], each memory item is a short natural language record with metadata (time, type, source, etc.). At inference time, the system retrieves the most relevant items and formats them as a compact memory block that is appended to the dialogue context, keeping only a handful of focused sentences in the prompt. Taking a more structured approach, A-MEM [157] organizes interaction history as Zettelkasten-style notes and uses a two-stage retrieval pipeline to select only a few high-utility notes; these notes are linearized into a small "working set" section inside the agent prompt, while the rest of the note graph remains offline. ACE [185] goes one step further and treats the agent context as an evolving playbook: it maintains a library of fine-grained strategy bullets with usage statistics, and before each episode it selects and injects only the most helpful bullets into the system instructions and memory prompts. Similarly, for execution efficiency, agentic plan caching [186] caches high-level plan templates distilled from successful past executions; at

serving time, a cheap keyword-based matcher looks up a matching template and a small planner LLM adapts it to the new query, replacing a fresh planning phase with a short plan-adaptation prompt. Finally, apart from structured storage, general compression techniques are also employed to fit external information into the prompt. RECOMP [153] uses Retrieve–Compress–Prepend: an extractive compressor selects sentences and an abstractive compressor writes a short summary, which is prepended to the query; selective augmentation allows returning an empty string when retrieval is unhelpful.

Across these methods, textual memory integration improves efficiency by compressing long histories into task-specific snippets that fit into the prompt while retaining the main signals that drive agent behavior.

Latent Integration. Latent memory integration stores long-term information as compact hidden states or key–value pairs and reuses them within the model's internal computation, avoiding re-encoding the original text.

One approach to latent integration is to scale latent memory capacity while keeping the GPU KV cache roughly constant. MemoryLLM [133] inserts a trainable pool of memory tokens into every transformer layer. During inference these tokens are processed together with the normal sequence tokens, so information stored in the memory pool can influence the hidden states at each step without extending the visible context. Based on MemoryLLM, M+ [134] adds a CPU-resident long-term memory and a co-trained retriever that fetches a small set of relevant hidden-state memory tokens per layer during generation, enabling long-range recall with similar GPU memory overhead.

Alternatively, some latent integration methods maintain external knowledge or long context directly as compressed KV-level states, which are then integrated into the generation process via attention. Memory^3 [165] stores a KB as explicit key–value memory and, during decoding, retrieves a few entries per token block and adds their KVs to the attention KV cache, avoiding long prompts. MemoRAG [94] compresses long context into a KV-cache global memory over inserted memory tokens; a lightweight memory model generates a draft answer as a retrieval clue, This design reduces the query-time long-context cost by running full long-context inference on only a few selected passages, while the rest of the corpus is accessed through compressed KV-level memory.

Compared with purely textual integration, these latent mechanisms push most long-term information into fixed-size neural states and expose them through attention, so that the cost of using long-horizon experience grows much more slowly than the length of the raw interaction history.

3.4. Multi-Agent Memory

In LLM-based multi-agent systems (MAS), many early studies, such as CAMEL [56], mainly focus on textual communication protocols, where memory can typically be regarded as implicit and implemented in a simple form. More recent research has begun to explicitly focus on the notion of memory in MAS. These memory-oriented works still fit into the taxonomy proposed in our framework, but in this section we adopt a MAS-centered perspective and provide a more focused discussion of memory within multi-agent systems.

Shared Memory. Shared memory centralizes reusable information across agents to mitigate redundancy, as duplicating multi-agent interaction histories in individual prompts is costly in both token budget and inference time. MS [24] stores agent steps as Prompt–Answer pairs and filters them with an LLM evaluator before adding them to a shared pool, then uses accepted memories to continually refine the retriever. However, the

frequent LLM-based scoring introduces substantial token and latency overhead.

To improve efficiency, recent work explores **structured shared textual memory** that supports lightweight retrieval and reduces redundant context replay. G-Memory [180] models multi-agent experience as a three-tier graph hierarchy of insight, query, and interaction graphs; at inference, it performs bi-directional traversal to retrieve high-level, generalizable insights together with fine-grained, condensed interaction trajectories for agent-specific working memory. RCR-Router [69] maintains a Shared Memory Store of interaction history, task-relevant knowledge, and structured state representations, and performs round-wise context routing with an Importance Scorer, a Semantic Filter, and a Token Budget Allocator to minimize redundant context and token usage. MemIndex [104] adopts an intent-indexed bipartite graph architecture for memory operations in LM-based multi-agent pub/sub systems, improving storage, retrieval, update, and deletion efficiency and reporting lower elapsed time, CPU utilization, and memory usage. Different from typical shared-memory MAS that mainly consume retrieved context, MIRIX [132] adopts a modular multi-agent architecture governed by a Meta Memory Manager and six Memory Managers, and uses Active Retrieval to generate a topic and inject retrieved memories into the system prompt without explicit memory-search prompts.

Beyond textual shared memory, **latent shared memory** enables agents to exchange compact internal states, reducing redundant token-level replay. LatentMAS [204] implements latent shared memory by having each agent perform auto-regressive latent thinking from last-layer hidden states and consolidating the resulting layer-wise KV caches into a shared latent working memory for persistent read–write sharing across agents. KVComm [174] enables training-free online KV-cache communication by maintaining an anchor pool of shared segments and their KV offsets, then matching anchors and approximating offsets to safely reuse KV caches across new prefixes, avoiding repeated prefilling.

Shared Memory

- **Advantages**: Enables cross-agent reuse of verified facts and decisions, improving coordination and efficiency by reducing redundant work and retries.
- **Disadvantages**: Prone to inconsistency from concurrent writes, and can become noisy and costly to retrieve without consolidation and access control.

Local Memory. For local memory, redundancy accumulates within each agent as its personal store grows, so retrieval and updates should remain agent-local; meanwhile, local memory management can borrow ideas from single-agent methods such as selective writing, consolidation, and capacity control. Intrinsic Memory Agents [177] equips each agent with a role-aligned structured memory template and updates it every turn by folding the agent's latest output back into the same template until consensus is reached. AgentNet [167] maintains fixed-size memory modules for the router and executor, and uses dynamic memory management with signals like frequency, recency, and uniqueness to prune low-utility trajectories at capacity. DAMCS [164] introduces A-KGMS, consolidating experiences into a goal-oriented hierarchical knowledge graph and planning via neighborhood queries around the most recent goal node to avoid full-history sharing and reduce overhead.

Local Memory

- **Advantages**: Lightweight, low-noise per-agent workspace that supports efficient retrieval and role-specific prompting.
- **Disadvantages**: Not shared across agents, so useful results may not propagate and work can be duplicated.

Mixed Memory. Mixed memory combines shared and local memory, and its efficiency often benefits from coordination between the two, including what to write to each, when to retrieve from which, and how to control redundancy. SRMT [102] couples each agent's personal memory vector with a shared recurrent memory by pooling all agents' memory vectors and letting agents cross-attend to this shared sequence, then updating their personal vectors via a memory head. Collaborative Memory [100] uses dynamic bipartite access graphs with private/shared tiers, storing fragments with immutable provenance and enforcing sharing through configurable read/write policies. LEGOMem [31] builds modular procedural memory with full-task memories for the orchestrator and subtask memories for task agents, comparing vanilla retrieval with Dynamic and QueryRewrite variants for finer-grained subtask memory access.

Mixed Memory

- **Advantages**: Combines efficient per-agent local state with cross-agent knowledge reuse via shared memory, improving both specialization and coordination.
- **Disadvantages**: Adds synchronization and routing complexity, and can still suffer from inconsistency or noise in the shared store.

3.5. Discussion

Trade-off Between Memory Compression and Performance. Although we have repeatedly emphasized that memory extraction can reduce costs such as input token usage, an unavoidable issue is that extraction may lead to the loss of critical information, which can directly degrade the agent's performance. This problem has also been noted in prior work such as AgentFold [175]. LightMem [21], for instance, explicitly takes the compression rate into account. Its experimental results clearly show that excessive compression leads to poorer accuracy, whereas milder compression better preserves performance but incurs relatively higher cost. Therefore, how to strike an appropriate balance between compression and performance remains an open question, and there may also be alternative approaches that aim to retain as much salient information as possible during the extraction or compression process.

Online vs Offline Memory Management. Regarding memory management strategies, A-MEM[157] exemplifies a purely online system where memory updates occur synchronously during interaction. As demonstrated by MemoryOS[49], such real-time updates incur frequent LLM calls per response, leading to higher latency and financial costs. By contrast, LightMem [21] adopts a hybrid architecture combining a lightweight online cache with offline consolidation. This design offloads expensive computations to asynchronous offline processes, significantly reducing inference time while maintaining similar overall computational costs. This comparison highlights a fundamental trade-off: online updates ensure immediate adaptation but increase latency and cost, whereas offline updates minimize inference overhead but suffer

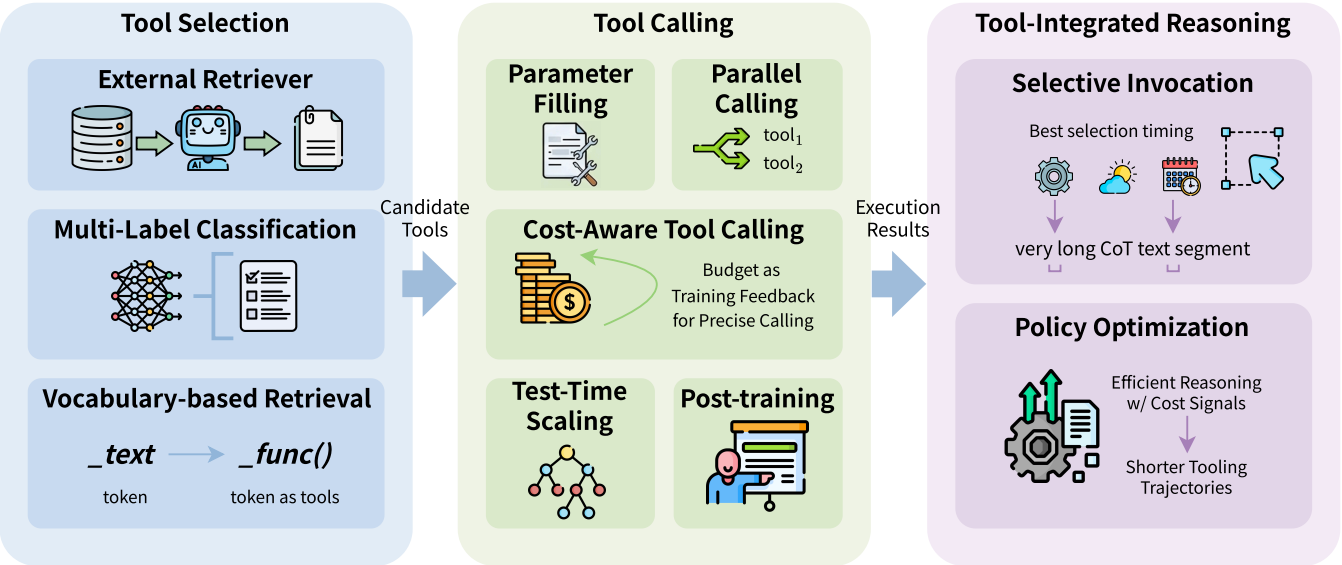


Figure 4: Efficient tool learning comprises three stages: Tool Selection identifies candidate tools via retrieval or classification; Tool Calling handles parameter filling and execution with a focus on cost-aware constraints and budget feedback and Tool-Integrated Reasoning optimizes efficient reasoning trajectories through selective invocation and policy optimization.

from slower adaptation. Consequently, this comparison suggests that an optimal memory system design should likely strike a balance between these two paradigms.

4. Efficient Tool Learning

Tool learning provides an interface for LLMs to interact with the physical world and virtual environment. In general, tools refer to search, code sandbox (interpreter), and many other general API endpoints. To call these tools, a basic solution is to provide several candidates to the prompt, and let the LLM think and select the most suitable one with parameters filled [172]. However, as the task become more complex, there would be much more tool calls. For example, LLMs may call the search API for 600 times to resolve a deep research problem [120]. Such long trajectories extremely challenge the models' long context comprehension ability and brings enormous costs. To this end, it is crucial to explore efficient tool learning strategies.

Overall, there are two types of efficiency in tool learning: (1) Tool learning itself is efficient for solving complex problems. Comparing with a task with very long CoT, tool learning could efficiently optimize the length of trajectories and show the efficient reasoning process. (2) Tool learning could be optimized to call fewer tools, which reduces the cost of tool learning itself. For a complex task with hundreds of tool calls, an optimal method could significantly reduces the number of tool calls. So the overall process would be even more efficient.

As shown in Figure 4, we introduce efficient tool learning in three main categories, including Tool Selection, Tool Calling, and Tool-Integrated Reasoning. Candidate tools are first selected to let LLM judge when and what to call, then the tool call's results would be embeded into the response and the reasoning trajectories.

Table 2: A summary of representative efficient tooling methods. We categorize them by tool selection, tool calling, and tool-integrated reasoning.

Method	Category	Core Mechanism	Resource Link
		Efficient Tool Selection	
ProTIP [4]	External Retriever	Contrastive learning to correlate queries with tools	N/A
TinyAgent [19]	Multi-Label Classification	Implement a small model to select appropriate tools	GitHub
Tool2Vec [81]	Multi-Label Classification	Align tools with synthetic usage examples	GitHub
ToolkenGPT [33]	Vocabulary-based Retrieval	Train tools as a special token	GitHub
Toolken+ [160]	Vocabulary-based Retrieval	Rerank top-k tools and reject if no one is selected	N/A
Chain-of-Tools [149]	Vocabulary-based Retrieval	Leverage CoT with a huge tool pool	GitHub
ToolGen [128]	Vocabulary-based Retrieval	Encode each tool as a separate token	GitHub
		Efficient Tool Calling	
Toolformer [106]	In-Place Parameter Filling	Leverage CoT to invoke tool calls	N/A
CoA [25]	In-Place Parameter Filling	Uses symbolic abstractions for intermediate steps	N/A
LLMCompiler [51]	Parallel Tool Calling	A compiler-inspired framework enabling parallel tooling	GitHub
LLM-Tool Compiler [111]	Parallel Tool Calling	Fusing similar tools and parallel tooling	N/A
CATP-LLM [145]	Parallel Tool Calling	Include cost-awareness into planning	GitHub
BTP [193]	Cost-Aware Tool Calling	Formulates tool calling as a knapsack problem	GitHub
TROVE [138]	Cost-Aware Tool Calling	Introduce compact reusable tools.	GitHub
ToolCoder [17]	Cost-Aware Tool Calling	Treat tool as code generation	GitHub
ToolChain* [203]	Test-Time Scaling	Utilizes A* search to prune unproductive branches	N/A
OTC-PO [125]	Post-training / RL	Integrates tool-use penalty into RL objective	N/A
ToolOrchestra [114]	Post-training / RL	Efficiency-aware rewards for specialized orchestrators	GitHub
		Tool-Integrated Reasoning (TIR)	
TableMind [45]	Adaptive Search	Plan-action-reflect loop with Rank-Aware Optimization	GitHub
SMART [93]	Boundary Awareness	CoT-based dataset to decide parametric vs. tool use	GitHub
ARTIST [110]	Policy Optimization	Unified agentic reasoning with outcome-based RL	N/A
AutoTIR [142]	Policy Optimization	Hybrid reward for correctness and format adherence	GitHub
ReTool [22]	Code-Integrated Reasoning	Dynamic NL-code interleaving with verifiable rewards	GitHub
ToolRL [92]	Structured Rewards	Combines format reward with tool parameter correctness	GitHub
PORTool [146]	Step-wise Planning	Uses fork-relative advantages and decay factors	N/A
Agent-FLAN [14]	Data Efficiency	Decomposes agent data into capability-specific subsets	GitHub

4.1. Tool Selection

For massive tool candidates from a very large pool, it is nearly impossible to stuff the prompt with thousands of tool descriptions. To this end, it is crucial to efficiently select the most relevant tools for user queries. We organize current tool retrieval literature into three categories: (1) **External Retriever**: a independent retriever model which embeds user queries and tool descriptions and calculates the affinity scores (e.g. cosine similarity) to select top-k relevant tools as candidates; (2) **Multi-Label Classification**: for a fixed size of tool sets, the tool selection process could be formulated as a multi-label classification problem, which directly predicts relevant tools; and (3) **Vocabulary-based Retrieval**: tools are embedded as special tokens into the model's vocabulary, and the model would enter a tool call mode when generating such tool tokens. We introduce the above three categories of tool selection strategies in this section as below.

External Retriever. Instead of including the entire tool set, many approaches rely on an external retriever for tool selection. External tool retrieval can be improved from retriever-side advances that redesign the retrieval pipeline or strengthen retrievers and rerankers, and from tool-side enhancements that refine tool descriptions and documentation to make the retrieval corpus easier to match, boosting both accuracy and

efficiency.

On the retriever side, ProTIP [4] utilizes a contrastive learning-based method to embed user queries and tool descriptions into the semantic space. After a tool is selected, ProTIP subtracts the query embedding by the selected tool's representation and selects tools on other subtasks. Such a progressive design makes ProTIP efficient to avoid explicit task decomposition overhead. In AnyTool [18], retrieval is organized hierarchically and inspired by a divide-and-conquer strategy, narrowing the search space and thereby improving retrieval efficiency.

On the tool side, DRAFT [97] refines tool documents via self-driven interactions to improve external tool retrieval, while boosting efficiency by reducing token overhead and stopping refinement at convergence.

In addition, some recent systems combine both directions. Toolshed [77] stores enriched tool representations in a tool knowledge base and uses RAG-tool fusion before, during, and after retrieval to scale external tool selection, while controlling top-k to curb token growth and improve efficiency. Similarly, ToolScope [70] uses ToolScopeMerger with Auto-Correction to compress tool descriptions and reduce input tokens, and ToolScopeRetriever to hybrid-retrieve top-k tools that fit the LLM context window, improving tool-use quality while boosting efficiency and scalability.

Multi-Label Classification (MLC). Instead of ranking-based retrieval, MLC-based methods treat tool selection as a classification task. TinyAgent [19] is designed to conduct tool calling on edge devices which pursue extreme efficiency, and it formulates the tool selection task as a multi-label classification problem. For a user query, TinyAgent applies the DeBERTa-v3 small model as the encoder and output the probability distribution for all available tools. Tools with a probability higher than 50% are recognized as relevant ones and will be selected accordingly. Since only a small fraction of tool descriptions are put into the prompt, it efficiently reduces nearly half of the prompt size. Similar to TinyAgent, Moon et al. [81] find MLC-based tool retrieval efficient, but such a task formulation could not handle the growing number of tools, and any updates would require a model re-training. Therefore, they propose Tool2Vec, a two-stage retrieval with a reranker for analyzing fine-grained tool-query interactions. To fill in the semantic gap where natural user queries may not directly align with tool descriptions, the authors generate tool embeddings based on synthetic usage examples rather than static description.

Vocabulary-based Retrieval. Besides direct retrieving from candidates by external retriever and MLC, tool selection could also be formulated as a token prediction task, where tools are stored in the vocabulary as special tokens.

ToolkenGPT [33] regards massive external tools as learnable token embeddings (aka. "toolken"), so that the target tool could be selected as a normal next token prediction process. Compared with Toolformer [106] that selects tools by predicting a whole trajectory with special characters, this approach is highly efficient since it only trains the added tool embeddings and keeps other model parameters frozen. Furthermore, it bypasses the window constraint of in-context tool selection and retains a shorter prompt. Building on this foundation, Toolken+ [160] enhances ToolkenGPT by introducing an extra reranking step and a rejection toolken, which improves the overall performance and reduces the hallucination rate. Toolken+ also demonstrates a tradeoff between efficiency and efficacy, which could be simply tuned from the number of reranking candidates. Although "toolkens" are efficient for massive tool selection, it requires constructing data samples for supervised fine-tuning and suffers from generalization problems on unseen tools. Similarly, ToolGen [128] assigns each tool a unique tool token and trains the model to turn tool retrieval and calling

into a unified generation task. By representing a tool with a single token, it is claimed to shorten generation and potentially reduce inference overhead but may be costly at training phase. From a different perspective on efficiency, Xu et al. [158] proposes selective compression and block compression for tool use: they preserve key information (e.g., tool and parameter names) as raw text while compressing the remaining documentation into fixed-length soft tokens per block. The soft tokens can be precomputed and cached offline, reducing prompt length and improving token efficiency at inference. To tackle the generalization problem, CoTools [149] shrinks the number of toolkens to only one and applies a retriever to calculate the similarities between current toolken's representation and all the candidates.

From these literature, we find vocabulary-based methods are an efficient option for tool selection. However, it may suffer from inaccurate invocation timing and poor generalization to unseen new tools, which is less functional for extensive tool updating scenarios.

Tool Selection

- **Advantages**: External retriever, MLC, and vocab-based methods are very efficient especially for retrieval from massive candidates. External retriever could be a plug-and-play module with a good generalization ability to unseen tools.
- **Disadvantages**: The external retriever may be a large model with more computational overhead than MLC and vocab-based tool tokens, while MLC and vocab-based tool retrieval may need fine-tuning to adapt models on new tools.
- **Applicable scenarios**: According to the types of candidate tools, if the candidate pool changes a lot over time, it would be better to use external retrievers. However, if the candidate tool set is relatively fixed, MLC and vocab-based methods are good options for better efficiency.

4.2. Tool Calling

Once candidates are selected, the efficiency of the invocation process becomes critical for real-time agentic interactions.

In-Place Parameter Filling. In-place tool calling is a paradigm where the model directly fills the tool's parameters during the response generation process. Toolformer [106] incorporates tool calling within the CoT path, and fills parameters during the response generation process. It is efficient to obtain the final results once the closure of the tool call is reached. Gao et al. [25] proposes CoA, which shares the similar idea but reduces the response time by providing more accurate tool call results. Instead of directly calculating the final results, CoA introduces symbolic abstractions to represent the intermediate steps, which are later substituted with the actual results during the response generation process. From the experimental results, CoA performs better while reducing more than 30% inference time than Toolformer.

Parallel Tool Calling. For a complex tasks that incorporates multiple tools, traditional sequential style calling may hurt efficiency since LLMs have to wait for the latest tool call's response. However, there are multiple tasks that could be done in parallel [188]. For example, to get the weather information of a province, we do not have to call the `get_weather` API one-by-one for each city. Instead, a more practical way is to make parallel tool calls, which would significantly reduce the overall task solving time. LLMCompiler [51] introduces a compiler-inspired framework that formulates execution plans, dispatches tasks, and executes functions in parallel. This achieves improvements in latency, cost, and overall accuracy against the traditional

sequential tool execution approach. Building on this parallelization paradigm, LLM-Tool Compiler [111] further optimizes efficiency by selectively fusing similar tool operations at runtime, which increases parallel tool calls while reducing token consumption and latency. Complementing with the above methods, CATP-LLM [145] addresses the execution cost by incorporating cost-awareness into the planning process. It designs a multi-branch planning language and employs cost-aware offline reinforcement learning to fine-tune models, enabling high-quality generation with economic constraints.

Cost-Aware Tool Calling. Like we have introduced about CATP-LLM in the above paragraph, cost could be a special reward for training efficient tool calling models. Budget-Constrained Tool Learning with Planning (BTP) [193] first formulates tool calling as a knapsack problem, which utilizes dynamic programming to pre-compute how often each tool would be invoked under a hard budget, thereby turning cost control into a forward-looking plan. Building on this planning strategy, Xu et al. [155] estimates LLM confidence via consistency-based sampling strategy to let the model trigger a tool under a certainty-cost optimal condition. This method could reduces the number of tool calls, thereby boosting the overall improvements. From a broader system perspective, Wu et al. [143] reduces redundant calls by jointly updating prompt strategy and tool documentations. It complements the above cost-aware planning and confidence-based gating with context-level efficiency.

Beyond directly constraining invocation budgets, recent research also explores improving efficiency through alternative paradigms, such as function induction, code generation, and model distillation. TROVE [138] introduces a training-free paradigm that incrementally builds and trims a compact toolbox of reusable functions, showing that online induction can improves the accuracy without extra training data. ToolCoder [17] extends this idea by formulating tool learning as an end-to-end code generation task, which converts tasks in natural languages into Python code. This method boosts the success rates while keeping small API usage cost. Focusing on the deployment cost, Kang et al. [50] proposes to distill LLM's knowledge into small language models with retrieval and code interpreter tools, which enables small models competitive with larger ones.

Efficient Test-Time Scaling. For effective tool calling, a viable solution is tree search-based strategies, where the model may plan a tree of tool calls and select the most promising path [150]. However, such methods are computationally expensive since they may need trail-and-error to explore the entire tree. Instead of extensive tree traversal, ToolChain* [203] utilizes the A* search strategy to efficiently navigate complex action spaces. This method boosts the efficiency by employing task-specific cost functions to prune wrong branches earlier and only requires single-step node expansions. Therefore, it allows the agent to prioritize the most promising paths and avoids exhaustive searches, leading to high success rates.

Efficient Tool Calling with Post-training. To mitigate the high latency and computational overhead with multi-step tool interactions, recent research has increasingly focused on optimizing tool-calling efficiency through post-training. Specifically, reinforcement learning has emerged as a primary mechanism for teaching models to strategically balance task success with resource parsimony. OTC-PO [125] promotes action-level efficiency by integrating a tool-use penalty into the reinforcement learning objective, which effectively trains models to minimize redundant tool calls without sacrificing answer correctness. Building on the optimization of agentic workflows, ToolOrchestra [114] leverages efficiency-aware rewards within an RL framework to train specialized orchestrators that achieve superior task performance at a fraction of the computational cost of general-purpose large language models. Complementing these strategy-driven approaches, ToolRM [1] addresses the challenge of precise evaluation by utilizing specialized outcome-based reward models to

facilitate data-efficient fine-tuning and inference-time scaling, ensuring that models learn to prioritize the most effective and concise tool-calling trajectories.

Tool Calling

- **Advantages**: The above tool calling methods focus on different aspects and could be applied simultaneously for better efficiency. Overall, for one trajectory, in-place parameter filling, cost-aware tool calling, test-time scaling, and post training with cost rewards are effective to improve the efficiency, while parallel tool calling could split one trajectory into different branches and finish calling in parallel.
- **Disadvantages**: Although test-time scaling could improve the tool calling accuracy and reduce the length of trajectories, it is still a trade-off between efficacy and efficiency. Besides, parallel tool calling may result in iterative refinement if the parallel task planner fails to find the task dependencies.
- **Applicable scenarios**: If the agent is in a plan-act-reflection mode that plans the whole tool calling trajectory instead of iterative refinement, parallel tool calling is a suitable option to split branches in advance. Besides, cost-aware tool calling and post-training methods are good strategies to reduce the number of tool calls. Efficient test-time scaling is a good way to increase the task accomplishment accuracy, therefore reducing the tool calling trajectories. While it may generate more tokens to try more branches, it is an applicable strategy to generate accurate trajectories for distillation.

4.3. Tool-Integrated Reasoning

The emergence of agents marks a crucial shift from reliance on static internal knowledge toward adaptive, multi-turn reasoning, which is necessary for achieving both high accuracy and computational efficiency in complex problem-solving [78, 101, 98]. Traditional, rigid programmatic workflows or purely text-based methods often fail on tasks requiring numerical precision or dynamic adaptation, thereby constraining the development of truly autonomous reasoning capabilities.

Selective Invocation. The quest for efficient agents begins with establishing a robust capability to invoke tools only when strictly necessary, thereby minimizing redundant computations. Traditional rigid workflows often lead to excessive interactions. The TableMind framework [45] addresses this by presenting an autonomous programmatic agent specifically tailored for tool-augmented table reasoning. Architecturally, TableMind utilizes an iterative plan-action-reflect loop, where the agent first decomposes a problem, then generates and executes precise code within a secure sandbox environment. TableMind employs a two-stage training paradigm: Supervised Fine-Tuning (SFT) serves as a vital warm-up phase to establish foundational tool usage patterns and master the necessary syntax for the iterative cycle, thereby mitigating the instability associated with starting subsequent Reinforcement Learning from a cold policy. To further refine the efficiency of tool invocation, Qian et al. [93] first constructs a dataset called SMART with CoT detailing the necessity of each tool call, and they use the dataset to fine-tune a model that efficiently decides whether to use their parametric knowledge or external tools. Agent-FLAN [14] separates format-following agent data from general reasoning data and further decomposes agent data into capability-specific subsets, which improves performance with fewer training tokens.

Cost-Aware Policy Optimization. Beyond supervised warm-up, Reinforcement Learning (RL) is pivotal for optimizing complex multi-step policies to ensure high reasoning quality and strict adherence to formatting

constraints. To prioritize high-quality trajectories, TableMind [45] employs the Rank-Aware Policy Optimization (RAPO) algorithm. RAPO identifies misaligned trajectories and applies rank-aware advantage weighting to guide the model toward consistent answers. In terms of strategic autonomy, the ARTIST framework [110] tightly couples agentic reasoning with outcome-based RL, enabling models to learn optimal tool-use strategies without restrictive step-level supervision. Similarly, ReTool [22] integrates a code interpreter directly into the reasoning loop, allowing the model to dynamically interleave natural language with executable code and discover strategies via verifiable reward signals. To further ensure the validity of these actions, ToolRL [92] designs a reward function that combines a format reward with a correctness reward, matching tool parameters against ground truth to improve success rates per call.

Concurrently, another research aspect focuses on making agents faster and more cost-effective by minimizing unnecessary tool invocations and reducing trajectories. Methods like A^2 FM [12] and IKEA [43] aim to balance internal knowledge with external retrieval. A^2 FM utilizes Adaptive Policy Optimization (APO) with a self-adaptive router to decide whether to answer instantly or invoke tools, while IKEA trains an adaptive search agent to rely on internal knowledge first and call search APIs only when necessary. To explicitly penalize redundancy, Wei et al. [142] introduce AutoTIR, which discourages unnecessary tool usage through specific reward penalties. Similarly, Wang et al. [125] leverage the OTC-PO algorithm to encourage trajectories with correct answers and fewer tool calls. Other approaches optimize the trajectory generation process itself. SWiRL [28] filters redundant actions during parallel trajectory generation, and PORTool [146] employs a decay factor γ to emphasize steps closer to the final outcome, favoring solutions that solve problems in fewer tool-call steps.

Tool-Integrated Reasoning

- **Advantages**: Tool-integrated reasoning strategies incorporate tool calls into the long reasoning path, which boosts the task accuracy. By invoking tools at suitable timings, TIR is very data-efficient to reduce the overall training samples.
- **Disadvantages**: Specific tools need special environments, which increases the system design. For example, coding agents needs sandbox environments to verify the generated code, which brings significant development complexity.
- **Applicable scenarios**: For a complex task that should invoke external resources (e.g. browsers, search APIs, and code interpreters), TIR is a good choice to interact with a real environment to accomplish tasks with multi-hop reasoning. For simple tasks that mainly depend on model's internal knowledge, TIR may be less efficient and brings additional tool calling costs.

4.4. Discussion

The evolution of efficient tooling reflects a fundamental shift from merely "enabling" tool use to "optimizing" the interaction loop. While efficient selection and calling techniques (e.g., retrieval and parallelism) address the structural bottlenecks of large toolsets and sequential latency, Tool-Integrated Reasoning targets the strategic overhead of the agent's decision-making process. The frontier of this field is moving toward a Pareto optimization of performance and cost: rather than maximizing tool usage for accuracy, modern agents are increasingly trained via RL to minimize redundant interactions. This transition suggests that future efficiency gains will likely stem from a tighter coupling between the model's internal reasoning and the external tool environment, where "acting" is no longer a separate step but an integrated, cost-aware component of the model's cognitive architecture.

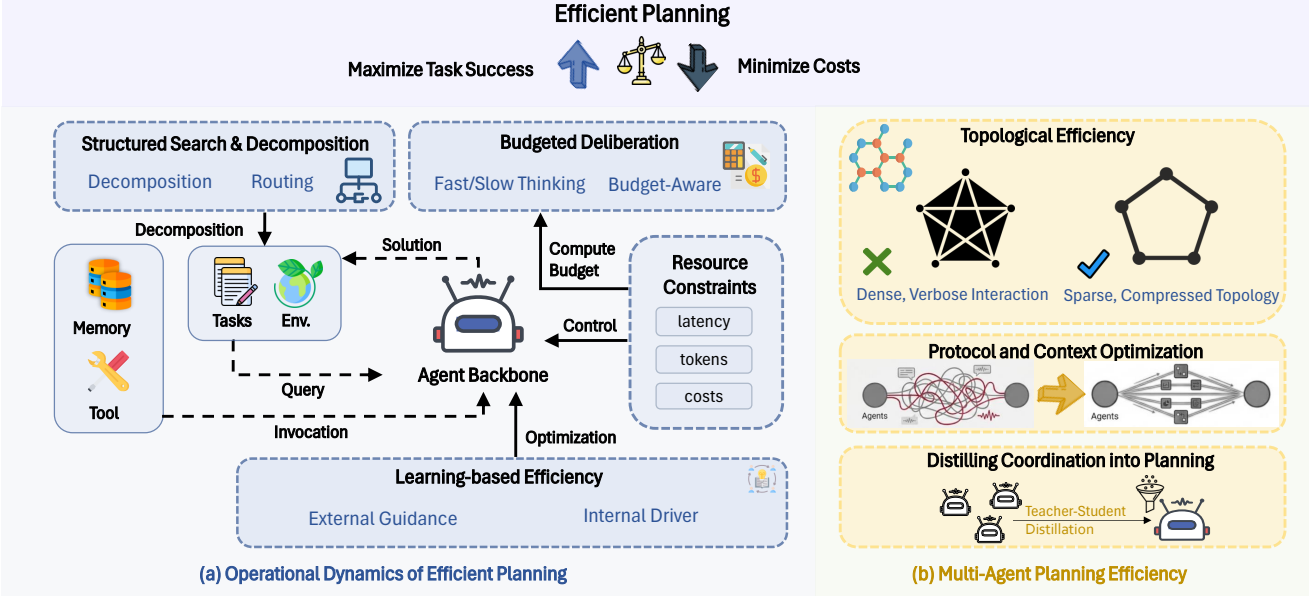


Figure 5: **Overview of Efficient Planning.** It aims to maximize task success while minimizing costs. **(a)** Single-agent methods optimize inference strategies (control, search, decomposition) or evolve via learning (policy, memory). **(b)** Multi-agent methods reduce overhead via topological optimization, context optimization, and coordination distillation.

5. Efficient Planning

Efficient Planning

- **Core Philosophy**: Frames deliberation as a resource-constrained control problem rather than unbounded reasoning.
- **Mechanism**: Optimizes the *depth* of single-agent reasoning (via search and learning) and the *breadth* of multi-agent collaboration (via topology and protocol).
- **Objectives**: Maximizes task success constraints on latency, token consumption, and communication overhead.

This perspective represents a distinct shift from classical planning, which assumes abundant computational resources, and contemporary approaches that conflate planning with direct text generation. Instead, efficient planning conceptualizes reasoning as *operational control*, where an agent must continuously balance the marginal utility of a refined plan against its computational cost. Within a broader architecture, the planner acts as the central engine for online compute allocation, synergizing with memory components to amortize costs and tools to externalize execution. In this section, we survey the landscape of efficient planning through two primary paradigms: **Single-Agent Planning**, which optimizes individual deliberation trajectories, and **Multi-Agent Collaborative Planning**, which minimizes the coordination overhead in distributed systems.

5.1. Single-Agent Planning Efficiency

Single-agent efficiency focuses on minimizing the computational cost, measured in tokens, latency, or search steps, required to reach a valid solution. We categorize these methods into *inference-time strategies*, which

Table 3: A summary of representative efficient planning methods. We categorize Single-Agent methods into **Inference-Time Strategy** (Adaptive Control, Search, Decomposition) and **Learning-based Evolution** (Policy, Memory), alongside **Multi-Agent Collaborative Efficiency**.

Method	Category	Core Mechanism	Resource Link
Single-Agent: Inference-Time Strategy (Search & Control)			
SwiftSage [64]	Adaptive Control	Fast/Slow Dual-process (System 1 + 2)	GitHub
Budget-Aware [72]	Adaptive Control	Budget-constrained tool policy allocation	N/A
Reflexion [109]	Adaptive Control	Verbal reinforcement from prior failures	GitHub
LATS [195]	Tree Search	MCTS with self-reflection	GitHub
ToolChain* [203]	Tree Search	A* search with learned cost pruning	N/A
CATS [191]	Tree Search	Cost-aware pruning in tree search	N/A
ReWOO [152]	Decomposition	Planner-Worker-Solver separation	GitHub
HuggingGPT [108]	Decomposition	Routing tasks to specialized models	GitHub
Alita [96]	Decomposition	MCP brainstorming & subtasking	GitHub
Single-Agent: Learning-based Evolution (Policy & Memory)			
QLASS [67]	Policy Optimization	Q-Value critic for search guidance	N/A
ETO [113]	Policy Optimization	Trial-and-error preference learning (DPO)	GitHub
VOYAGER [124]	Memory & Skill	Iterative skill library construction	GitHub
GAP [147]	Memory & Skill	Graph-based decomposition & parallelism	GitHub
RLTR [62]	Policy Optimization	Process-level reward training	N/A
Planning w/o Search [36]	Policy Optimization	Offline goal-conditioned critic	Website
Multi-Agent: Collaborative Efficiency			
Chain-of-Agents [189]	Topology	Sequential context passing (Linear complexity)	GitHub
MacNet [91]	Topology	DAG-based topological ordering	GitHub
AgentPrune [179]	Topology	Learned pruning of communication edges	GitHub
MARS [129]	Topology	Reviewer-Meta-Reviewer pipeline (No debate)	GitHub
CodeAgents [163]	Protocol	Structured pseudocode interaction	GitHub
Free-MAD [16]	Protocol	Prompt-optimized critical reasoning	N/A
MAGDI [10]	Distillation	Distilling interaction graphs into student	GitHub
D&R [199]	Distillation	Distilling debate traces via DPO	N/A

optimize the planning process on-the-fly, and *learning-based evolution*, which improves the agent's intrinsic planning capabilities.

Inference Strategy I: Adaptive Budgeting and Control. A key strategy is *selective deliberation*, allocating computational effort non-uniformly. Architectures like SwiftSage [64] separate fast behaviors from slower planning, defaulting to heuristics unless structured reasoning is required. This can be framed as learning when to invoke a costly planner versus a reactive policy [85], or dynamically adjusting tool strategies based on budget constraints [72]. Efficiency is also gained by preventing redundant failures; methods like Reflexion [109] and ReST [2] use verbal reinforcement or iterative refinement to amortize failure analysis, lowering cumulative interaction costs.

Inference Strategy II: Structured Search. The combinatorial explosion of action spaces presents a significant bottleneck. To address this, methods adapt formal search algorithms to prune feasible trajectories.

Language Agent Tree Search (LATS) [195] reframes agent rollouts as Monte Carlo Tree Search, enabling self-reflection to guide exploration. Building on this, CATS [191] integrates cost-awareness directly into the search tree, pruning expensive branches early. In tool-rich environments, ToolChain* [203] applies A* search to navigate the action space, while retrieval-based approaches like ProTIP [4] reduce decision complexity by only surfacing relevant tools during the planning phase.

Inference Strategy III: Task Decomposition. Explicitly breaking down complex tasks reduces context overhead. ReWOO [152] and Alita [96] decouple planning from execution, generating blueprints to avoid step-by-step token redundancy. This decomposition facilitates routing: HuggingGPT [108] and ReSo [196] dispatch sub-tasks to specialized models, while BudgetMLAgent [23] optimizes agent routing for cost. In embodied settings, AutoGPT+P [8] grounds this planning in environmental affordances to ensure feasibility.

Learning-Based Evolution: Policy Optimization. Agents can learn to internalize planning logic. This is driven by external critics, such as QLASS [67] or offline value functions [36], that guide the planner toward high-value actions. Alternatively, learning acts as an *internal driver*: ETO [113] refines policies via trial-and-error preference learning (DPO). To improve sample efficiency, methods like RLTR [62] and Planner-R1 [202] utilize process-level rewards, providing feedback on the reasoning sequence rather than just the final outcome.

Learning-Based Evolution: Memory and Skill Acquisition. Efficiency can be amortized by externalizing successful plans. VOYAGER [124] builds a library of reusable skills to avoid re-planning. Graph-based representations also support this: GraphReader [60] and other graph-enhanced models [65] leverage structured memory for long-context retrieval, while GAP [147] identifies parallelizable actions. Ultimately, frameworks like Sibyl [135] demonstrate that efficiency is an emergent property, where improved memory structure directly reduces the cognitive load of future planning.

Single-Agent Strategies

- **Advantages**: Adaptive control lowers inference cost, structured search improves exploration efficiency, task decomposition reduces step-by-step redundancy and context overhead, and learning-based evolution amortizes planning cost over time.
- **Disadvantages**: Adaptive control can misfire, structured search introduces overhead, task decomposition risks error propagation, and learning and memory add training and maintenance cost.

5.2. Multi-Agent Collaborative Efficiency

Multi-agent systems (MAS) offer enhanced reasoning but often incur quadratic communication costs. Efficient MAS planning therefore focuses on optimizing the *topology* of interaction and the *content* of protocols.

Topological Efficiency and Sparsification. Topological efficiency optimizes the communication graph, mitigating quadratic message costs and reducing message complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ through structured topologies (e.g., chains, DAGs). *Structured topologies* like Chain-of-Agents [189] and MacNet [91] restrict context growth to near-linear complexity, while GroupDebate [73] alternates between dense debate and sparse summaries. *Selective interaction* protocols further filter turns; MARS [129] and S^2-MAD [178]

eliminate direct peer-to-peer noise by only triggering debates when viewpoints diverge. More advanced methods, such as AgentPrune [179], AgentDropout [137], and SafeSieve [187], dynamically learn to prune low-utility edges or progressively sparsify the graph during inference.

Protocol and Context Optimization. Protocol optimization improves efficiency by compressing what is communicated, using concise representations such as pseudocode and prompt-driven constraints to reduce interaction context. CodeAgents [163] encodes reasoning in concise pseudocode, while Smurfs [9] discards failed search branches to prevent context bloat. In parallel, prompt-level control accelerates convergence; Free-MAD [16] and ConsensAgent [90] engineer prompts to encourage critical reasoning, while supervisors like SMAS [66] terminate redundant loops early.

Distilling Coordination into Planning. The most radical approach internalizes coordination by distilling collective intelligence into a single-agent model, bypassing runtime coordination costs. Methods like MAGDI [10] and SMAGDi [3] distill complex interaction graphs or "Socratic" decomposition into a single student model. Similarly, D&R [199] uses a teacher-student debate to generate preference trees for DPO. These approaches retain the quality benefits of diverse perspectives while reverting to the lower inference cost of a single agent.

Multi-Agent Strategies

- **Advantages**: Topology sparsification reduces communication cost, protocol compression prevents context bloat, and coordination distillation keeps quality while lowering inference cost.
- **Disadvantages**: Pruning may drop useful signals, compression may lose key details, and distillation adds training cost and can weaken diversity at inference time.

5.3. Discussion

Efficient agent planning reframes reasoning from an unbounded generation process into a budget-aware control problem. In the **single-agent** regime, we observe a clear taxonomy of *inference-time strategies*, ranging from adaptive budgeting to structured search, and *learning-based evolution* that amortizes cost via policy refinement and skill memory. In the **multi-agent** regime, the focus shifts to topological pruning and the distillation of collective intelligence. Across both, the unifying trend is the migration of computation from *online search* to *offline learning* or *structured retrieval*, enabling agents to achieve complex goals within strict resource constraints.

6. Benchmarks

Although this survey focuses on efficiency, we adopt an effectiveness-first view: a method that is cheap but fails to solve tasks or substantially harms solution quality is not meaningfully efficient. Accordingly, we characterize efficiency in two complementary ways: comparing effectiveness under a fixed cost budget, or comparing cost at a comparable level of effectiveness. This trade-off can also be viewed through the Pareto frontier between effectiveness and cost. We provide a high-level overview of benchmarks for memory, tool learning, and planning.

6.1. Memory

Effectiveness Benchmarks. Agent memory effectiveness is commonly evaluated either indirectly via downstream end-to-end task success or directly via memory-targeted tasks [190]. For indirect end-to-end evaluation, agent-memory research typically measures downstream task outcomes, using both QA datasets like HotpotQA [168] and Natural Questions [52] and interactive agent benchmarks where success requires multi-step interaction and tool use, such as GAIA [80]. In addition, a growing body of work has proposed benchmarks that directly evaluate agent memory abilities, such as LoCoMo [79] and LongMemEval [144].

Efficiency Benchmarks. Beyond effectiveness, Some memory benchmarks additionally report efficiency-related signals. Evo-Memory [141] introduces step efficiency, measuring how many environment steps are required to reach a goal; fewer steps indicate that the memory mechanism supports more concise and scalable reasoning. StoryBench [122] reports two auxiliary efficiency metrics: Runtime Cost, capturing the total time required to run the memory-augmented agent on long-horizon tasks, and Token Consumption, which serves as a proxy for how much contextual information the model processes. MemBench [117] explicitly incorporates temporal efficiency into its evaluation, reporting read time and write time (seconds per memory operation) to approximate the overhead introduced by different memory designs and to highlight configurations whose processing time may be prohibitive in practical deployments.

Efficiency Metrics in Memory Methods. At the methods level, we summarize efficiency metrics reported by existing memory mechanisms, grouped into four categories. Existing work adopts various metrics to quantify the efficiency of memory mechanisms in LLM-based agents. Overall, these metrics can be summarized into four categories.

Token consumption and API cost are among the most frequently used signals. Many studies report efficiency in terms of token consumption [153, 60, 40, 86, 15, 157, 49, 200]. Beyond raw token counts, some approaches further translate token usage into monetary cost (USD), such as Agentic Plan Caching [186] and ACE [185].

Time-based metrics focus on latency and runtime overhead. HiAgent [40] reports overall runtime, while SeCom [86], Mem0 [15], and MemOS [63] measure end-to-end latency that combines search time and LLM reasoning but explicitly excludes construction time. MEM1 [200] reports inference time. Other work targets finer-grained retrieval overhead, e.g., A-MEM [157], H-MEM [115], and Agent KB [119] measure retrieval time or search latency. MemoRAG [94] further distinguishes index latency from retrieval latency, which denotes the time taken to fetch evidence for a query.

Resource-based metrics quantify hardware consumption. For example, A-MEM [157] and MemoRAG [94] report GPU memory usage, and MemoRAG additionally analyzes GPU memory consumption under different context lengths.

Finally, interaction-based metrics capture how intensively the agent interacts with the LLM or a reasoning process. MemoryOS [49] reports the average number of LLM calls per response, while benchmarks such as ReasoningBank [83] track the number of reasoning steps as an interaction-level efficiency indicator.

6.2. Tool Learning

Tool learning still lacks unified efficiency benchmarks, and most evaluations prioritize effectiveness. Yet it is crucial for LLM agents because tool use often dominates interaction cost and drives end-to-end success. We

summarize three benchmark families: selection and parameter infilling, tool learning under model context protocols, and agentic tool learning. These testbeds also support efficiency measurement using tokens, latency, and tool-call turns.

Benchmarks for Selection & Parameter Infilling. Tool construction is an important dimension to assess, since the availability and quality of tools can fundamentally shape downstream tool-use behavior. Seal-Tools [148] applies LLMs to efficiently generate large scale tools and use cases. While it includes multi-tool instances (with some nested tool callings), more complex compositions can still be under-represented compared to benchmarks that center multi-tool orchestration. UltraTool [41] starts from user queries in real scenarios and evaluate models on tool creation tasks.

Once tools are available, an agent must learn whether to use tools and which tools to select. MetaTool [42] specifically focuses on the decision-making process of whether to employ a tool and which tool to select from a candidate set. The study assess models across diverse scenarios, including reliability issues and multi-tool requirements.

After selecting tools, parameter infilling and schema adherence become the next challenge for reliable execution. Berkeley Function-Calling Leaderboard (BFCL) [88] is a series of benchmarks for tool learning evaluation, which includes tools for real applications with multi-turn and multi-step dialogues. API-Bank [59] provides a manually annotated benchmark with 73 tools, which is more natural for common dialogues.

Beyond single-tool usage, many realistic tasks require multi-tool composition, including sequential and nested tool calls, making long-horizon coordination an essential capability to evaluate. NesTools [32] categorizes multi-tool calling problem and provides comprehensive taxonomy for nested tool learning. τ-Bench [173] is a simple tool learning benchmark for retail and airline domains, and τ^2-Bench [6] further extends it with the telecom domain, which incorporates tool calls from the users. ToolBench [95] is a large scale dataset which collects more than 16,000 APIs from RapidAPI[1] . However, the online API service is not always stable and there are reproducibility issues in ToolBench. To address this gap between training instructions and real-user queries, Wu et al. [150] proposes MGToolBench, where they manually curated the ToolBench dataset with multiple granularities.

However, end-to-end success metrics alone often make it difficult to localize where and why failures occur within the tool-use process. T-Eval [13] introduces a fine-grained benchmark for evaluating tool utilization by decomposing the process into six constituent capabilities, including planning, reasoning, and retrieval, to enable step-by-step assessment rather than relying solely on holistic outcome metrics.

Finally, from a system-level perspective, some methods focus on the reproducibility and efficiency of evaluation, which is particularly important when benchmarks depend on real-world online APIs. Guo et al. [30] introduce StableToolBench, a benchmark that ensures consistent assessment through a virtual API server employing caching and LLM-based simulation alongside a robust GPT-4 evaluation framework.

Tool Learning with Model Context Protocol. As tool learning becomes a common practice for LLM agent development, Anthropic proposes Model Context Protocol (MCP) [2] to provide a standard for tool definitions and calling. Based on this protocol, several benchmarks are proposed. MCP-RADAR [26] explicitly evaluates efficiency via metrics such as tool selection efficiency, computational resource efficiency, and execution speed, alongside accuracy-related dimensions. MCP-Bench [136] evaluates agent efficiency with an LLM-as-a-Judge

[1] https://rapidapi.com/

[2] https://www.anthropic.com/news/model-context-protocol

rubric, where the parallelism and efficiency criterion scores whether the agent minimizes redundant tool calls and exploits opportunities for parallel execution.

Agentic Tool Learning. With the growing capabilities of tool learning, modern LLMs gradually saturate on general tool learning benchmarks, and the community requires new evaluation criteria. To this end, instead of tool selection and parameter infilling, modern agentic tool learning tasks mainly focus on how to answer a very complex or unpopular question with iteratively calling search APIs. SimpleQA [139] is designed to evaluate the ability of LLMs to provide factually correct short answers. The benchmark is challenging for frontier models with high correctness and covers a diverse range of topics. BrowseComp [140] largely follows SimpleQA to let human trainers create challenging questions with short and verifiable answers. Since these questions are hard to answer by models' internal knowledge, they would heavily rely on the browsing ability, which correlated to the search tool. SealQA [89] is another benchmark that evaluates search-augmented LLMs on fact-seeking questions where web search results are likely to be conflicting, noisy, and unhelpful. The most challenging 111 questions are composed to be a subset of SEAL-0, where even frontier models consistently achieve near-zero accuracy.

6.3. Planning

Effectiveness Benchmarks. Based on tool learning benchmarks, many benchmarks further consider planning into consideration. Planning effectiveness is often assessed indirectly via downstream task success in agent benchmarks, such as SWE-Bench [47], WebArena [198], and WebShop [170]. While prior work like PlanBench [121] has introduced evaluations of planning efficiency, most existing planning benchmarks focus on pure LLM settings, which limits their direct applicability to LLM-based agents. In agentic systems, efficiency depends not only on LLM-side compute (e.g., token usage), but also on closed-loop interaction costs (e.g., environment steps and tool calls). Therefore, alongside the rapid progress in agent effectiveness, planning efficiency has become an equally important evaluation dimension; in what follows, we summarize the benchmarks and metrics used to quantify it.

Efficiency Benchmarks. Several recent works have begun to emerge in this direction. Based on Blocksworld domain [112], Jobs et al. [48] proposes a structured benchmark to evaluate an agent's planning and execution. From an efficiency perspective, it reports end-to-end execution time, the number of planning attempts, token consumption, and the corresponding monetary cost. TPS-Bench [154] evaluates not only effectiveness but also planning and tooling efficiency using token usage, end-to-end time, and tool-call turns. It further proposes cost-of-pass, the expected monetary cost per successful completion, linking token-based cost with completion rate for cost-effectiveness comparison across models. CostBench [68] benchmarks cost-optimal tool-use planning under dynamic changes. It models tools with explicit costs and evaluates efficiency via Cost Gap and path deviation from the ground-truth trajectory, while also accounting for invalid tool calls as tooling inefficiency.

Efficiency Metrics in Planning Methods. Apart from benchmarks, many agent planning methods also evaluate the efficiency of their approaches. Consistent with the metrics discussed earlier, they also consider token consumption [64, 195, 152, 196] and runtime [203].

From the perspective of search depth and breadth, SwiftSage [64] considers time steps, Reflexion [109] counts the number of trials, LATS computes the lowest average number of nodes/states required for success,

and CATS [191] reports the average number of iterations needed to find a valid solution. This aspect is strongly related to planning efficiency.

In addition, cost-of-pass style metrics [20] have also been adopted, as seen in TPS-Bench [154] and subsequent work [127, 147]. Beyond directly using cost-of-pass, many methods further operationalize it in evaluation. A common practice is to keep the budget the same and compare performance, such as [72, 67, 113].

7. Challenges and Future Directions

Towards a unified efficiency evaluation framework for agent memory. Although these methods and benchmarks all attempt to quantify memory efficiency, they do so using different subsets of the above dimensions and heterogeneous terminology. Some works only report token consumption or API cost, while others focus on runtime, latency, inference time, retrieval time, or step efficiency, often without clearly specifying which stages of the pipeline are included. Even token-based metrics may be defined per query, per memory operation, per episode, or for constructing the memory store. As a result, existing efficiency numbers are not directly comparable across papers, which makes it difficult to systematically analyze the cost–performance trade-offs of different memory designs.

Agentic Latent Reasoning. Recent months have seen growing interest in latent-space reasoning for LLM[131, 58, 159] , where intermediate computations are carried out in continuous hidden representations rather than being fully externalized as natural-language tokens. Compared with token-level "decode-and-read" reasoning, latent reasoning can reduce token overhead and may preserve richer, high-dimensional information during multi-step computation. However, existing work has largely focused on standalone LLM settings, while agentic latent reasoning remains relatively underexplored. This gap is important because agentic scenarios introduce additional requirements, such as tool use, long-horizon planning, memory management, and action verification, that differ from pure text-only reasoning and may demand new training objectives, interfaces, and evaluation protocols. Investigating latent reasoning mechanisms tailored for agents could therefore be a promising future direction.

Deployment-Aware Agentic Design. Inspired by MemAgent [176] and Chain-of-Agents [189], which address long-context reasoning by chunking context and processing it sequentially, we argue that agentic systems should be more deployment-aware. In practice, multi-agent designs can be realized either as true multi-model deployments or as single-model role-play pipelines, and these implementations differ substantially in orchestration overhead, latency, and reliability. Future work should compare these alternatives under matched resource budgets and report end-to-end cost–benefit metrics, clarifying whether the performance gains from adding more agents justify the additional complexity.

Efficiency Challenges and Directions for MLLM-based Agents. There has been a rapid emergence of MLLM-based agent methods, including agents equipped with multimodal memory [61, 130, 105], approaches that explicitly enhance planning and decision-making for MLLM-based agents [74], and multi-agent systems built upon LLM and MLLM backbones [164, 132], among others. However, efficiency in MLLM-based agents is relatively under-explored, which is also emphasized in [169]. In realistic deployments, efficiency is crucial due to the need for rapid responses under strict latency and compute budgets. In this regard, we observe that several efficiency techniques in LLM-based agents may inspire MLLM-based agents. For

example, SwiftSage [64] adopts fast–slow mode switching to allocate computation adaptively, and FAST-GRPO [151] explores a similar fast–slow thinking mechanism for MLLM agents. Nevertheless, transferring text-centric efficiency strategies to multimodal agents remains challenging. Compared with language-only settings, MLLM-based agents often operate in different action spaces and task structures, such as GUI-based or embodied interactions, while multimodal perception and grounding can introduce additional latency and compound errors over long-horizon interactions [35]. Notably, long-horizon multimodal tasks require maintaining a visual history. The cumulative computational burden of re-encoding visual context for every step creates a trade-off between memory retention and inference speed that is far more severe than in LLM-based agents. As a future direction, we advocate efficiency-aware agent design and evaluation for MLLM-based agents by jointly considering performance and cost, including latency, interaction steps, and tool-call overhead.

8. Conclusion

In conclusion, this survey summarizes the evolution from LLMs to LLM-based agents, highlighting the shift toward increasingly complex settings that motivates our discussion. We review three core components, *memory*, *tool learning*, and *planning*, with an emphasis on efficiency, and find that many seemingly different methods converge on shared high-level ideas. We also summarize efficiency-oriented benchmarks and commonly used metrics across both benchmark and methodological studies. Finally, we outline key challenges and future directions. Overall, our survey consolidates the design space and evaluation practices for agent efficiency, while underscoring the need for more standardized and transparent reporting to enable fair comparison and reproducibility. We hope this survey offers useful guidance for designing and evaluating efficient agents, and helps encourage further progress in this direction.

References

[1] Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A. Lastras, Yara Rizk, and Pavan Kapanipathi. Toolrm: Outcome reward models for tool-calling large language models, 2025. URL https://arxiv.org/abs/2509.11963.

[2] Renat Aksitov, Sobhan Miryoosefi, Zonglin Li, Daliang Li, Sheila Babayan, Kavya Kopparapu, Zachary Fisher, Ruiqi Guo, Sushant Prakash, Pranesh Srinivasan, Manzil Zaheer, Felix Yu, and Sanjiv Kumar. Rest meets react: Self-improvement for multi-step reasoning llm agent, 2023. URL https://arxiv.org/abs/2312.10003.

[3] Aayush Aluru, Myra Malik, Samarth Patankar, Spencer Kim, Kevin Zhu, Sean O'Brien, and Vasu Sharma. Smagdi: Socratic multi agent interaction graph distillation for efficient high accuracy reasoning. *arXiv preprint arXiv:2511.05528*, 2025.

[4] Raviteja Anantha, Bortik Bandyopadhyay, Anirudh Kashi, Sayantan Mahinder, Andrew W Hill, and Srinivas Chappidi. Protip: Progressive tool retrieval improves planning, 2023. URL https://arxiv.org/abs/2312.10332.

[5] Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Andrey Kravchenko, Mikhail Burtsev, and Evgeny Burnaev. Arigraph: Learning knowledge graph world models with episodic memory for llm agents. *arXiv preprint arXiv:2407.04363*, 2024.

[6] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment, 2025. URL https://arxiv.org/abs/2506.07982.

[7] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.

[8] Timo Birr, Christoph Pohl, Abdelrahman Younes, and Tamim Asfour. Autogpt+p: Affordance-based task planning with large language models, 2024. URL https://arxiv.org/abs/2402.10778.

[9] Junzhi Chen, Juhao Liang, and Benyou Wang. Smurfs: Multi-agent system using context-efficient dfsdt for tool planning. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3281–3298, 2025.

[10] Justin Chih-Yao Chen, Swarnadeep Saha, Elias Stengel-Eskin, and Mohit Bansal. Magdi: Structured distillation of multi-agent interaction graphs improves reasoning in smaller language models. *arXiv preprint arXiv:2402.01620*, 2024.

[11] Nuo Chen, Hongguang Li, Jianhui Chang, Juhua Huang, Baoyuan Wang, and Jia Li. Compress to impress: Unleashing the potential of compressive memory in real-world long-term conversations. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 755–773, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL https://aclanthology.org/2025.coling-main.51/.

[12] Qianben Chen, Jingyi Cao, Jiayu Zhang, Tianrui Qin, Xiaowan Li, King Zhu, Dingfeng Shi, He Zhu, Minghao Liu, Xiaobo Liang, Xin Gui, Ge Zhang, Jian Yang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. A^2fm: An adaptive agent foundation model for tool-aware hybrid reasoning, 2025. URL https://arxiv.org/abs/2510.12838.

[13] Zehui Chen, Weihua Du, Wenwei Zhang, Kuikun Liu, Jiangning Liu, Miao Zheng, Jingming Zhuo, Songyang Zhang, Dahua Lin, Kai Chen, et al. T-eval: Evaluating the tool utilization capability of large language models step by step. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9510–9529, 2024.

[14] Zehui Chen, Kuikun Liu, Qiuchen Wang, Wenwei Zhang, Jiangning Liu, Dahua Lin, Kai Chen, and Feng Zhao. Agent-flan: Designing data and methods of effective agent tuning for large language models, 2024. URL https://arxiv.org/abs/2403.12881.

[15] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[16] Yu Cui, Hang Fu, Haibin Zhang, Licheng Wang, and Cong Zuo. Free-mad: Consensus-free multi-agent debate. *arXiv preprint arXiv:2509.11035*, 2025.

[17] Hanxing Ding, Shuchang Tao, Liang Pang, Zihao Wei, Jinyang Gao, Bolin Ding, Huawei Shen, and Xueqi Cheng. Toolcoder: A systematic code-empowered tool learning framework for large language models, 2025. URL https://arxiv.org/abs/2502.11404.

[18] Yu Du, Fangyun Wei, and Hongyang Zhang. AnyTool: Self-reflective, hierarchical agents for large-scale API calls. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11812–11829. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/du24h.html.

[19] Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, Sehoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Richard Charles Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Tinyagent: Function calling at the edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 80–88, 2024.

[20] Mehmet Hamza Erol, Batu El, Mirac Suzgun, Mert Yuksekgonul, and James Zou. Cost-of-pass: An economic framework for evaluating language models. *arXiv preprint arXiv:2504.13359*, 2025.

[21] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, et al. Lightmem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*, 2025.

[22] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms, 2025. URL https://arxiv.org/abs/2504.11536.

[23] Shubham Gandhi, Manasi Patwardhan, Lovekesh Vig, and Gautam Shroff. Budgetmlagent: A cost-effective llm multi-agent system for automating machine learning tasks, 2025. URL https://arxiv.org/abs/2411.07464.

[24] Hang Gao and Yongfeng Zhang. Memory sharing for large language model based agents. *arXiv preprint arXiv:2404.09982*, 2024.

[25] Silin Gao, Jane Dwivedi-Yu, Ping Yu, Xiaoqing Ellen Tan, Ramakanth Pasunuru, Olga Golovneva, Koustuv Sinha, Asli Celikyilmaz, Antoine Bosselut, and Tianlu Wang. Efficient tool use with chain-of-abstraction reasoning, 2025. URL https://arxiv.org/abs/2401.17464.

[26] Xuanqi Gao, Siyi Xie, Juan Zhai, Shiqing Ma, and Chao Shen. Mcp-radar: A multi-dimensional benchmark for evaluating tool use capabilities in large language models, 2025. URL https://arxiv.org/abs/2505.16700.

[27] Felix A Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. *Neural computation*, 12(10):2451–2471, 2000.

[28] Anna Goldie, Azalia Mirhoseini, Hao Zhou, Irene Cai, and Christopher D. Manning. Synthetic data generation & multi-step rl for reasoning & tool use, 2025. URL https://arxiv.org/abs/2504.04736.

[29] J Gottweis, WH Weng, A Daryin, T Tu, A Palepu, P Sirkovic, and V Natarajan. Towards an ai co-scientist: A multi-agent system for scientific discovery. *arXiv preprint arXiv:2502.18864*, page 3, 2025.

[30] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models, 2025. URL https://arxiv.org/abs/2403.07714.

[31] Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. Legomem: Modular procedural memory for multi-agent llm systems for workflow automation. *arXiv preprint arXiv:2510.04851*, 2025.

[32] Han Han, Tong Zhu, Xiang Zhang, Mengsong Wu, Hao Xiong, and Wenliang Chen. Nestools: A dataset for evaluating nested tool learning abilities of large language models, 2025. URL https://arxiv.org/abs/2410.11805.

[33] Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 45870–45894. Curran Associates, Inc., 2023.

[34] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.371. URL https://aclanthology.org/2024.acl-long.371/.

[35] Zefeng He, Xiaoye Qu, Yafu Li, Tong Zhu, Siyuan Huang, and Yu Cheng. Diffthinker: Towards generative multimodal reasoning with diffusion models. *arXiv preprint arXiv:2512.24165*, 2025.

[36] Joey Hong, Anca Dragan, and Sergey Levine. Planning without search: Refining frontier llms with offline goal-conditioned rl, 2025. URL https://arxiv.org/abs/2505.18098.

[37] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2023.

[38] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14281–14290, 2024.

[39] Yuki Hou, Haruki Tamoto, and Homei Miyashita. " my agent understands me better": Integrating dynamic human-like memory recall and consolidation in llm-based agents. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–7, 2024.

[40] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32779–32798, 2025.

[41] Shijue Huang, Wanjun Zhong, Jianqiao Lu, Qi Zhu, Jiahui Gao, Weiwen Liu, Yutai Hou, Xingshan Zeng, Yasheng Wang, Lifeng Shang, Xin Jiang, Ruifeng Xu, and Qun Liu. Planning, creation, usage: Benchmarking llms for comprehensive tool utilization in real-world complex scenarios, 2024. URL https://arxiv.org/abs/2401.17167.

[42] Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, and Lichao Sun. Metatool benchmark for large language models: Deciding whether to use tools and which to use, 2024. URL https://arxiv.org/abs/2310.03128.

[43] Ziyang Huang, Xiaowei Yuan, Yiming Ju, Jun Zhao, and Kang Liu. Reinforced internal-external knowledge synergistic reasoning for efficient adaptive search agent. *arXiv preprint arXiv:2505.07596*, 2025.

[44] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

[45] Chuang Jiang, Mingyue Cheng, Xiaoyu Tao, Qingyang Mao, Jie Ouyang, and Qi Liu. Tablemind: An autonomous programmatic agent for tool-augmented table reasoning. *arXiv preprint arXiv:2509.06278*, 2025.

[46] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. KG-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9505–9523, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.468. URL https://aclanthology.org/2025.acl-long.468/.

[47] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[48] Niklas Jobs, Luis Miguel Vieira da Silva, Jayanth Somashekaraiah, Maximilian Weigand, David Kube, and Felix Gehlhoff. Benchmark for planning and control with large language model agents: Blocksworld with model context protocol. *arXiv preprint arXiv:2512.03955*, 2025.

[49] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25961–25970, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1318. URL https://aclanthology.org/2025.emnlp-main.1318/.

[50] Minki Kang, Jongwon Jeong, Seanie Lee, Jaewoong Cho, and Sung Ju Hwang. Distilling llm agent into small models with retrieval and code tools, 2025. URL https://arxiv.org/abs/2505.17612.

[51] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. In *Forty-first International Conference on Machine Learning*, 2024.

[52] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

[53] Yann LeCun and Yoshua Bengio. Convolutional networks for images, speech, and time series. *The handbook of brain theory and neural networks*, 1998.

[54] Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. In *Proceedings of the 41st International Conference on Machine Learning*, pages 26396–26415, 2024.

[55] Xiang Lei, Qin Li, and Min Zhang. D-smart: Enhancing llm dialogue consistency via dynamic structured memory and reasoning tree. *arXiv preprint arXiv:2510.13363*, 2025.

[56] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.

[57] Hao Li, Chenghao Yang, An Zhang, Yang Deng, Xiang Wang, and Tat-Seng Chua. Hello again! llm-powered personalized agent for long-term dialogue. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5259–5276, 2025.

[58] Hengli Li, Chenxi Li, Tong Wu, Xuekai Zhu, Yuxuan Wang, Zhaoxin Yu, Eric Hanchen Jiang, Song-Chun Zhu, Zixia Jia, Ying Nian Wu, and Zilong Zheng. Seek in the dark: Reasoning via test-time instance-level policy gradient in latent space, 2025. URL https://arxiv.org/abs/2505.13308.

[59] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.187. URL https://aclanthology.org/2023.emnlp-main.187/.

[60] Shilong Li, Yancheng He, Hangyu Guo, Xingyuan Bu, Ge Bai, Jie Liu, Jiaheng Liu, Xingwei Qu, Yangguang Li, Wanli Ouyang, Wenbo Su, and Bo Zheng. GraphReader: Building graph-based agent to enhance long-context abilities of large language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 12758–12786, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.746. URL https://aclanthology.org/2024.findings-emnlp.746/.

[61] Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Dongmei Jiang, and Liqiang Nie. Optimus-1: Hybrid multimodal memory empowered agents excel in long-horizon tasks. *Advances in neural information processing systems*, 37:49881–49913, 2024.

[62] Zhiwei Li, Yong Hu, and Wenqing Wang. Encouraging good processes without the need for good answers: Reinforcement learning for llm agent planning, 2025. URL https://arxiv.org/abs/2508.19598.

[63] Zhiyu Li, Shichao Song, Chenyang Xi, Hanyu Wang, Chen Tang, Simin Niu, Ding Chen, Jiawei Yang, Chunyu Li, Qingchen Yu, et al. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*, 2025.

[64] Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks, 2023. URL https://arxiv.org/abs/2305.17390.

[65] Fangru Lin, Emanuele La Malfa, Valentin Hofmann, Elle Michelle Yang, Anthony Cohn, and Janet B. Pierrehumbert. Graph-enhanced large language models in asynchronous plan reasoning, 2024. URL https://arxiv.org/abs/2402.02805.

[66] Fulin Lin, Shaowen Chen, Ruishan Fang, Hongwei Wang, and Tao Lin. Stop wasting your tokens: Towards efficient runtime multi-agent systems. *arXiv preprint arXiv:2510.26585*, 2025.

[67] Zongyu Lin, Yao Tang, Xingcheng Yao, Da Yin, Ziniu Hu, Yizhou Sun, and Kai-Wei Chang. Qlass: Boosting language agent inference via q-guided stepwise search. *arXiv preprint arXiv:2502.02584*, 2025.

[68] Jiayu Liu, Cheng Qian, Zhaochen Su, Qing Zong, Shijue Huang, Bingxiang He, and Yi R Fung. Costbench: Evaluating multi-turn cost-optimal planning and adaptation in dynamic environments for llm tool-use agents. *arXiv preprint arXiv:2511.02734*, 2025.

[69] Jun Liu, Zhenglun Kong, Changdi Yang, Fan Yang, Tianqi Li, Peiyan Dong, Joannah Nanjekye, Hao Tang, Geng Yuan, Wei Niu, et al. Rcr-router: Efficient role-aware context routing for multi-agent llm systems with structured memory. *arXiv preprint arXiv:2508.04903*, 2025.

[70] Marianne Menglin Liu, Daniel Garcia, Fjona Parllaku, Vikas Upadhyay, Syed Fahad Allam Shah, and Dan Roth. Toolscope: Enhancing llm agent tool use through tool merging and context-aware filtering, 2025. URL https://arxiv.org/abs/2510.20036.

[71] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[72] Tengxiao Liu, Zifeng Wang, Jin Miao, I-Hung Hsu, Jun Yan, Jiefeng Chen, Rujun Han, Fangyuan Xu, Yanfei Chen, Ke Jiang, Samira Daruki, Yi Liang, William Yang Wang, Tomas Pfister, and Chen-Yu Lee. Budget-aware tool-use enables effective agent scaling, 2025. URL https://arxiv.org/abs/2511.17006.

[73] Tongxuan Liu, Xingyu Wang, Weizhe Huang, Wenjiang Xu, Yuting Zeng, Lei Jiang, Hailong Yang, and Jing Li. Groupdebate: Enhancing the efficiency of multi-agent debate using group discussion. *arXiv preprint arXiv:2409.14051*, 2024.

[74] Yuhang Liu, Pengxiang Li, Zishu Wei, Congkai Xie, Xueyu Hu, Xinchen Xu, Shengyu Zhang, Xiaotian Han, Hongxia Yang, and Fei Wu. Infiguiagent: A multimodal generalist gui agent with native reasoning and reflection, 2025. URL https://arxiv.org/abs/2501.04575.

[75] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[76] Junru Lu, Siyu An, Mingbao Lin, Gabriele Pergola, Yulan He, Di Yin, Xing Sun, and Yunsheng Wu. Memochat: Tuning llms to use memos for consistent long-range open-domain conversation. *arXiv preprint arXiv:2308.08239*, 2023.

[77] Elias Lumer, Vamse Kumar Subbiah, James A Burke, Pradeep Honaganahalli Basavaraju, and Austin Huber. Toolshed: Scale tool-equipped agents with advanced rag-tool fusion and tool knowledge bases. *arXiv preprint arXiv:2410.14594*, 2024.

[78] Yubo Ma, Zhibin Gou, Junheng Hao, Ruochen Xu, Shuohang Wang, Liangming Pan, Yujiu Yang, Yixin Cao, Aixin Sun, Hany Awadalla, et al. Sciagent: Tool-augmented language models for scientific reasoning. *arXiv preprint arXiv:2402.11451*, 2024.

[79] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.747. URL https://aclanthology.org/2024.acl-long.747/.

[80] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2023.

[81] Suhong Moon, Siddharth Jha, Lutfi Eren Erdogan, Sehoon Kim, Woosang Lim, Kurt Keutzer, and Amir Gholami. Efficient and scalable estimation of tool representations in vector space. *arXiv preprint arXiv:2409.02141*, 2024.

[82] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[83] Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.

[84] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[85] Davide Paglieri, Bartłomiej Cupiał, Jonathan Cook, Ulyana Piterbarg, Jens Tuyls, Edward Grefenstette, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. Learning when to plan: Efficiently allocating test-time compute for llm agents, 2025. URL https://arxiv.org/abs/2509.03581.

[86] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H Vicky Zhao, Lili Qiu, et al. Secom: On memory construction and retrieval for personalized conversational agents. In *The Thirteenth International Conference on Learning Representations*, 2025.

[87] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.

[88] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.

[89] Thinh Pham, Nguyen Nguyen, Pratibha Zunjare, Weiyuan Chen, Yu-Min Tseng, and Tu Vu. Sealqa: Raising the bar for reasoning in search-augmented language models. *arXiv preprint arXiv:2506.01062*, 2025.

[90] Priya Pitre, Naren Ramakrishnan, and Xuan Wang. Consensagent: Towards efficient and effective consensus in multi-agent llm interactions through sycophancy mitigation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22112–22133, 2025.

[91] Chen Qian, Zihao Xie, Yifei Wang, Wei Liu, Kunlun Zhu, Hanchen Xia, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, et al. Scaling large language model-based multi-agent collaboration. *arXiv preprint arXiv:2406.07155*, 2024.

[92] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.

[93] Cheng Qian, Emre Can Acikgoz, Hongru Wang, Xiusi Chen, Avirup Sil, Dilek Hakkani-Tur, Gokhan Tur, and Heng Ji. Smart: Self-aware agent for tool overuse mitigation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4604–4621, 2025.

[94] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. Memorag: Boosting long context processing with global memory-enhanced retrieval augmentation. In *Proceedings of the ACM Web Conference 2025 (TheWebConf 2025)*, Sydney, Australia, 2025. ACM. URL https://arxiv.org/abs/2409.05591.

[95] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, dahai li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024.

[96] Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, et al. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution. *arXiv preprint arXiv:2505.20286*, 2025.

[97] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. From exploration to mastery: Enabling llms to master tools via self-driven interactions. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=QKBu1BOAwd.

[98] Xiaoye Qu, Yafu Li, Zhaochen Su, Weigao Sun, Jianhao Yan, Dongrui Liu, Ganqu Cui, Daizong Liu, Shuxian Liang, Junxian He, et al. A survey of efficient reasoning for large reasoning models: Language, multimodality, and beyond. *arXiv preprint arXiv:2503.21614*, 2025.

[99] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[100] Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. Collaborative memory: Multi-user memory sharing in llm agents with dynamic access control. *arXiv preprint arXiv:2505.18279*, 2025.

[101] Jingqing Ruan, Yihong Chen, Bin Zhang, Zhiwei Xu, Tianpeng Bao, Guoqing Du, Shiwei Shi, Hangyu Mao, Ziyue Li, Xingyu Zeng, et al. Tptu: large language model-based ai agents for task planning and tool usage. *arXiv preprint arXiv:2308.03427*, 2023.

[102] Alsu Sagirova, Yuri Kuratov, and Mikhail Burtsev. Srmt: shared memory for multi-agent lifelong pathfinding. *arXiv preprint arXiv:2501.13200*, 2025.

[103] Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. MemInsight: Autonomous memory augmentation for LLM agents. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33136–33152, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1683. URL https://aclanthology.org/2025.emnlp-main.1683/.

[104] Alaa Saleh, Sasu Tarkoma, Anders Lindgren, Praveen Kumar Donta, Schahram Dustdar, Susanna Pirttikangas, and Lauri Lovén. Memindex: Agentic event-based distributed memory management for multi-agent systems. *ACM Transactions on Autonomous and Adaptive Systems*, 2025.

[105] Gabriel Sarch, Lawrence Jang, Michael Tarr, William W Cohen, Kenneth Marino, and Katerina Fragkiadaki. Vlm agents generate their own memories: Distilling experience into embodied programs of thought. *Advances in Neural Information Processing Systems*, 37:75942–75985, 2024.

[106] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.

[107] Mike Schuster and Kuldip K Paliwal. Bidirectional recurrent neural networks. *IEEE transactions on Signal Processing*, 45(11):2673–2681, 1997.

[108] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face, 2023. URL https://arxiv.org/abs/2303.17580.

[109] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

[110] Joykirat Singh, Raghav Magazine, Yash Pandya, and Akshay Nambi. Agentic reasoning and tool integration for llms via reinforcement learning. *arXiv preprint arXiv:2505.01441*, 2025.

[111] Simranjit Singh, Andreas Karatzas, Michael Fore, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. An llm-tool compiler for fused parallel function calling, 2024. URL https://arxiv.org/abs/2405.17438.

[112] John Slaney and Sylvie Thiébaux. Blocks world revisited. *Artificial Intelligence*, 125(1-2):119–153, 2001.

[113] Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. Trial and error: Exploration-based trajectory optimization of LLM agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7584–7600, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.409. URL https://aclanthology.org/2024.acl-long.409/.

[114] Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, Yi Dong, Evelina Bakhturina, Tao Yu, Yejin Choi, Jan Kautz, and Pavlo Molchanov. Toolorchestra: Elevating intelligence via efficient model and tool orchestration, 2025. URL https://arxiv.org/abs/2511.21689.

[115] Haoran Sun and Shaoning Zeng. Hierarchical memory for high-efficiency long-term reasoning in llm agents. *arXiv preprint arXiv:2507.22925*, 2025.

[116] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952*, 2025.

[117] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. Membench: Towards more comprehensive evaluation on the memory of llm-based agents. *arXiv preprint arXiv:2506.21605*, 2025.

[118] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, et al. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8416–8439, 2025.

[119] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, et al. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229*, 2025.

[120] MiroMind Team, Song Bai, Lidong Bing, Carson Chen, Guanzheng Chen, Yuntao Chen, Zhe Chen, Ziyi Chen, Jifeng Dai, Xuan Dong, et al. Mirothinker: Pushing the performance boundaries of open-source research agents via model, context, and interactive scaling. *arXiv preprint arXiv:2511.11793*, 2025.

[121] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems*, 36:38975–38987, 2023.

[122] Luanbo Wan and Weizhi Ma. Storybench: A dynamic benchmark for evaluating long-term memory with multi turns. *arXiv preprint arXiv:2506.13356*, 2025.

[123] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, et al. Efficient large language models: A survey. *arXiv preprint arXiv:2312.03863*, 2023.

[124] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=ehfRiF0R3a.

[125] Hongru Wang, Cheng Qian, Wanjun Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. Acting less is reasoning more! teaching model to act efficiently, 2025. URL https://arxiv.org/abs/2504.14870.

[126] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

[127] Ningning Wang, Xavier Hu, Pai Liu, He Zhu, Yue Hou, Heyuan Huang, Shengyu Zhang, Jian Yang, Jiaheng Liu, Ge Zhang, et al. Efficient agents: Building effective agents while reducing cost. *arXiv preprint arXiv:2508.02694*, 2025.

[128] Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. Toolgen: Unified tool retrieval and calling via generation. *arXiv preprint arXiv:2410.03439*, 2024.

[129] Xiao Wang, Jia Wang, Yijie Wang, Pengtao Dang, Sha Cao, and Chi Zhang. Mars: toward more efficient multi-agent collaboration for llm reasoning. *arXiv preprint arXiv:2509.20502*, 2025.

[130] Xiaohan Wang, Yuhui Zhang, Orr Zohar, and Serena Yeung-Levy. Videoagent: Long-form video understanding with large language model as agent. In *European Conference on Computer Vision*, pages 58–76. Springer, 2024.

[131] Xiaoqiang Wang, Suyuchen Wang, Yun Zhu, and Bang Liu. System-1.5 reasoning: Traversal in language and latent spaces with dynamic shortcuts. *arXiv preprint arXiv:2505.18962*, 2025.

[132] Yu Wang and Xi Chen. Mirix: Multi-agent memory system for llm-based agents. *arXiv preprint arXiv:2507.07957*, 2025.

[133] Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, et al. Memoryllm: towards self-updatable large language models. In *Proceedings of the 41st International Conference on Machine Learning*, pages 50453–50466, 2024.

[134] Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. M+: Extending memoryLLM with scalable long-term memory. In *Forty-second International Conference on Machine Learning*, 2025.

[135] Yulong Wang, Tianhao Shen, Lifeng Liu, and Jian Xie. Sibyl: Simple yet effective agent framework for complex real-world reasoning, 2024. URL https://arxiv.org/abs/2407.10718.

[136] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, et al. Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers. *arXiv preprint arXiv:2508.20453*, 2025.

[137] Zhexuan Wang, Yutong Wang, Xuebo Liu, Liang Ding, Miao Zhang, Jie Liu, and Min Zhang. Agent-Dropout: Dynamic agent elimination for token-efficient and high-performance LLM-based multi-agent collaboration. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24013–24035, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1170. URL https://aclanthology.org/2025.acl-long.1170/.

[138] Zhiruo Wang, Daniel Fried, and Graham Neubig. Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks, 2024. URL https://arxiv.org/abs/2401.12869.

[139] Jason Wei, Nguyen Karina, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. Measuring short-form factuality in large language models. *arXiv preprint arXiv:2411.04368*, 2024.

[140] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

[141] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H Chi, et al. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025.

[142] Yifan Wei, Xiaoyan Yu, Yixuan Weng, Tengfei Pan, Angsheng Li, and Li Du. Autotir: Autonomous tools integrated reasoning via reinforcement learning. *arXiv preprint arXiv:2507.21836*, 2025.

[143] Bin Wu, Edgar Meij, and Emine Yilmaz. A joint optimization framework for enhancing efficiency of tool utilization in LLM agents. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22361–22373, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1149. URL https://aclanthology.org/2025.findings-acl.1149/.

[144] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

[145] Duo Wu, Jinghe Wang, Yuan Meng, Yanning Zhang, Le Sun, and Zhi Wang. Catp-llm: Empowering large language models for cost-aware tool planning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8699–8709, 2025.

[146] Feijie Wu, Weiwu Zhu, Yuxiang Zhang, Soumya Chatterjee, Jiarong Zhu, Fan Mo, Rodin Luo, and Jing Gao. Portool: Tool-use llm training with rewarded tree. *arXiv preprint arXiv:2510.26020*, 2025.

[147] Jiaqi Wu, Qinlao Zhao, Zefeng Chen, Kai Qin, Yifei Zhao, Xueqian Wang, and Yuhang Yao. Gap: Graph-based agent planning with parallel tool use and reinforcement learning. *arXiv preprint arXiv:2510.25320*, 2025.

[148] Mengsong Wu, Tong Zhu, Han Han, Chuanyuan Tan, Xiang Zhang, and Wenliang Chen. Seal-tools: Self-instruct tool learning dataset for agent tuning and detailed benchmark, 2024. URL https://arxiv.org/abs/2405.08355.

[149] Mengsong Wu, Tong Zhu, Han Han, Xiang Zhang, Wenbiao Shao, and Wenliang Chen. Chain-of-tools: Utilizing massive unseen tools in the cot reasoning of frozen language models. *arXiv preprint arXiv:2503.16779*, 2025.

[150] Qinzhuo Wu, Wei Liu, Jian Luan, and Bin Wang. ToolPlanner: A tool augmented LLM for multi granularity instructions with path planning and feedback. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 18315–18339, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.1018. URL https://aclanthology.org/2024.emnlp-main.1018/.

[151] Wenyi Xiao and Leilei Gan. Fast-slow thinking grpo for large vision-language model reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[152] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023.

[153] Fangyuan Xu, Weijia Shi, and Eunsol Choi. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *arXiv preprint arXiv:2310.04408*, 2023.

[154] Hanwen Xu, Xuyao Huang, Yuzhe Liu, Kai Yu, and Zhijie Deng. Tps-bench: Evaluating ai agents' tool planning\& scheduling abilities in compounding tasks. *arXiv preprint arXiv:2511.01527*, 2025.

[155] Hongshen Xu, Zihan Wang, Zichen Zhu, Lei Pan, Xingyu Chen, Shuai Fan, Lu Chen, and Kai Yu. Alignment for efficient tool calling of large language models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17787–17803, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.898. URL https://aclanthology.org/2025.emnlp-main.898/.

[156] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, Daliang Xu, Qipeng Wang, Bingyang Wu, Yihao Zhao, Chen Yang, Shihe Wang, et al. A survey of resource-efficient llm and multimodal foundation models. *arXiv preprint arXiv:2401.08092*, 2024.

[157] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. In *Advances in Neural Information Processing Systems*, 2025.

[158] Yang Xu, Yunlong Feng, Honglin Mu, Yutai Hou, Yitong Li, Xinghao Wang, Wanjun Zhong, Zhongyang Li, Dandan Tu, Qingfu Zhu, Min Zhang, and Wanxiang Che. Concise and precise context compression for tool-using language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 16430–16441, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.974. URL https://aclanthology.org/2024.findings-acl.974/.

[159] Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. Softcot: Soft chain-of-thought for efficient reasoning with llms, 2025. URL https://arxiv.org/abs/2502.12134.

[160] Konstantin Yakovlev, Sergey Nikolenko, and Andrey Bout. Toolken+: Improving LLM tool usage with reranking and a reject option. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 5967–5974, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.345. URL https://aclanthology.org/2024.findings-emnlp.345/.

[161] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.

[162] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Kristian Kersting, Jeff Z Pan, Hinrich Schütze, et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.

[163] Bruce Yang, Xinfeng He, Huan Gao, Yifan Cao, Xiaofan Li, and David Hsu. Codeagents: A token-efficient framework for codified multi-agent reasoning in llms. *arXiv preprint arXiv:2507.03254*, 2025.

[164] Hanqing Yang, Jingdi Chen, Marie Siew, Tania Lorido-Botran, and Carlee Joe-Wong. Llm-powered decentralized generative agents with adaptive hierarchical knowledge graph for cooperative planning. *arXiv preprint arXiv:2502.05453*, 2025.

[165] Hongkang Yang, Zehao Lin, Wenjin Wang, Hao Wu, Zhiyu Li, Bo Tang, Wenqiang Wei, Jinbo Wang, Zeyun Tang, Shichao Song, et al. Memory3: Language modeling with explicit memory. *arXiv preprint arXiv:2407.01178*, 2024.

[166] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

[167] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems. *arXiv preprint arXiv:2504.00587*, 2025.

[168] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380, 2018.

[169] Huanjin Yao, Ruifei Zhang, Jiaxing Huang, Jingyi Zhang, Yibo Wang, Bo Fang, Ruolin Zhu, Yongcheng Jing, Shunyu Liu, Guanbin Li, et al. A survey on agentic multimodal large language models. *arXiv preprint arXiv:2510.10991*, 2025.

[170] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.

[171] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

[172] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

[173] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL https://arxiv.org/abs/2406.12045.

[174] Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, et al. Kvcomm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *arXiv preprint arXiv:2510.12872*, 2025.

[175] Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, et al. Agentfold: Long-horizon web agents with proactive context management. *arXiv preprint arXiv:2510.24699*, 2025.

[176] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, et al. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.

[177] Sizhe Yuen, Francisco Gomez Medina, Ting Su, Yali Du, and Adam J Sobey. Intrinsic memory agents: Heterogeneous multi-agent llm systems through structured contextual memory. *arXiv preprint arXiv:2508.08997*, 2025.

[178] Yuting Zeng, Weizhe Huang, Lei Jiang, Tongxuan Liu, Xitai Jin, Chen Tianying Tiana, Jing Li, and Xiaohua Xu. S^2-mad: Breaking the token barrier to enhance multi-agent debate efficiency, 2025. URL https://arxiv.org/abs/2502.04790.

[179] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024.

[180] Guibin Zhang, Muxin Fu, Kun Wang, Guancheng Wan, Miao Yu, and Shuicheng YAN. G-memory: Tracing hierarchical memory for multi-agent systems. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=mmIAp3cVS0.

[181] Guibin Zhang, Muxin Fu, and Shuicheng Yan. Memgen: Weaving generative latent memory for self-evolving agents. *arXiv preprint arXiv:2509.24704*, 2025.

[182] Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, et al. A survey of reinforcement learning for large reasoning models. *arXiv preprint arXiv:2509.08827*, 2025.

[183] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. CodeAgent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13643–13658, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.737. URL https://aclanthology.org/2024.acl-long.737/.

[184] Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. Long context compression with activation beacon. *arXiv preprint arXiv:2401.03462*, 2024.

[185] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.

[186] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Cost-efficient serving of llm agents via test-time plan caching, 2025. URL https://arxiv.org/abs/2506.14852.

[187] Ruijia Zhang, Xinyan Zhao, Ruixiang Wang, Sigen Chen, Guibin Zhang, An Zhang, Kun Wang, and Qingsong Wen. Safesieve: From heuristics to experience in progressive pruning for llm-based multi-agent communication. *arXiv preprint arXiv:2508.11733*, 2025.

[188] Xiang Zhang, Tong Zhu, Han Han, Hao Pan, Mengsong Wu, and Wenliang Chen. Parallel task planning via model collaboration. In Xian-Ling Mao, Zhaochun Ren, and Muyun Yang, editors, *Natural Language Processing and Chinese Computing*, pages 79–91, Singapore, 2025. Springer Nature Singapore. ISBN 978-981-95-3352-7.

[189] Yusen Zhang, Ruoxi Sun, Yanfei Chen, Tomas Pfister, Rui Zhang, and Sercan Arik. Chain of agents: Large language models collaborating on long-context tasks. *Advances in Neural Information Processing Systems*, 37:132208–132237, 2024.

[190] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025.

[191] Zihao Zhang and Fei Liu. Cost-augmented monte carlo tree search for llm-assisted planning. *arXiv preprint arXiv:2505.14656*, 2025.

[192] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.

[193] Yuanhang Zheng, Peng Li, Ming Yan, Ji Zhang, Fei Huang, and Yang Liu. Budget-constrained tool learning with planning. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 9039–9052, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.536. URL https://aclanthology.org/2024.findings-acl.536/.

[194] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024.

[195] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models, 2024. URL https://arxiv.org/abs/2310.04406.

[196] Heng Zhou, Hejia Geng, Xiangyuan Xue, Li Kang, Yiran Qin, Zhiyong Wang, Zhenfei Yin, and Lei Bai. Reso: A reward-driven self-organizing llm-based multi-agent system for reasoning tasks, 2025. URL https://arxiv.org/abs/2503.02390.

[197] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning llm agents without fine-tuning llms. *arXiv preprint arXiv:2508.16153*, 2025.

[198] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023. URL https://webarena.dev.

[199] Xiaofeng Zhou, He-Yan Huang, and Lizi Liao. Debate, reflect, and distill: Multi-agent feedback with tree-structured preference optimization for efficient language model enhancement. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 9122–9137, 2025.

[200] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents, 2025. URL https://arxiv.org/abs/2506.15841.

[201] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024.

[202] Siyu Zhu, Yanbin Jiang, Hejian Sang, Shao Tang, Qingquan Song, Biao He, Rohit Jain, Zhipeng Wang, and Alborz Geramifard. Planner-r1: Reward shaping enables efficient agentic rl with smaller llms, 2025. URL https://arxiv.org/abs/2509.25779.

[203] Yuchen Zhuang, Xiang Chen, Tong Yu, Saayan Mitra, Victor Bursztyn, Ryan A. Rossi, Somdeb Sarkhel, and Chao Zhang. Toolchain*: Efficient action space navigation in large language models with a* search, 2023. URL https://arxiv.org/abs/2310.13227.

[204] Jiaru Zou, Xiyuan Yang, Ruizhong Qiu, Gaotang Li, Katherine Tieu, Pan Lu, Ke Shen, Hanghang Tong, Yejin Choi, Jingrui He, et al. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.